On Weak NIZKs, One-way Functions and Amplification

Suvradip Chakraborty* James Hulett† Dakshita Khurana‡

Abstract

An $(\epsilon_{\mathsf{s}}, \epsilon_{\mathsf{zk}})$-weak non-interactive zero knowledge (NIZK) argument has soundness error at most ϵ_{s} and zero-knowledge error at most ϵ_{zk}. We show that as long as NP is hard in the *worst case*, the existence of an $(\epsilon_{\mathsf{s}}, \epsilon_{\mathsf{zk}})$-weak NIZK proof or argument for NP with $\epsilon_{\mathsf{zk}} + \sqrt{\epsilon_{\mathsf{s}}} < 1$ implies the existence of one-way functions. To obtain this result, we introduce and analyze a strong version of universal approximation that may be of independent interest.

As an application, we obtain NIZK amplification theorems based on very mild worst-case complexity assumptions. Specifically, [Bitansky-Geier, CRYPTO'24] showed that $(\epsilon_{\mathsf{s}}, \epsilon_{\mathsf{zk}})$-weak NIZK proofs (with ϵ_{s} and ϵ_{zk} constants such that $\epsilon_{\mathsf{s}} + \epsilon_{\mathsf{zk}} < 1$) can be amplified to make their errors negligible, but needed to assume the existence of one-way functions. Our results can be used to remove the additional one-way function assumption and obtain NIZK amplification theorems that are (almost) unconditional; only requiring the mild worst-case assumption that if $\mathsf{NP} \subseteq \mathsf{ioP/poly}$, then $\mathsf{NP} \subseteq \mathsf{BPP}$.

*Visa Research. suvchakr@visa.com

†UIUC. jhulett2@illinois.edu

‡UIUC and NTT Research. dakshita@illinois.edu

Contents

1 Introduction

Zero-knowledge proofs are a cornerstone of modern cryptography. They allow a prover to convince a verifier about the validity of an NP statement without revealing any information about the witness. The fact that nothing is revealed is captured by demonstrating an efficient simulator that given only the the statement, can simulate a proof that is indistinguishable from a real proof.

An important research agenda, first initiated by Ostrovsky and Wigderson [Ost91, OW93], aims to understand the minimal cryptographic assumptions necessary for the existence of zero-knowledge proof systems. In particular, these works showed that if NP is *hard-on-average*, then the existence of zero-knowledge proofs for NP implies the existence of one-way functions. Very recently, the beautiful work of Hirahara and Nanashima [HN24] made a major improvement, proving that only the *worst-case hardness* of NP (specifically, $\mathsf{NP} \not\subseteq \mathsf{ioP/poly}$) suffices to obtain an implication from ZK for NP to one-way functions.

Importantly, all the above works consider zero-knowledge protocols that have negligible security errors. This work aims to understand the cryptographic hardness necessary to build *weaker* forms of zero-knowledge proof systems, namely, those that have non-negligible soundness and zero-knowledge errors. A key motivation for studying such weaker primitives is that they are often easier to construct, and could also end up requiring weaker cryptographic assumptions than their fully secure counterparts.

Weakly-secure NIZKs and One-way Functions. In this work, we focus on a specific type of zero-knowledge proof; namely non-interactive zero-knowledge (NIZK) for NP. NIZKs allow a prover to send a single (non-interactive) proof string that convinces a verifier about the validity of a statement, in a model where both the prover and verifier have access to a randomly sampled common reference string (CRS). The zero-knowledge property is captured by an efficient simulator that given only the statement, can output a CRS and proof pair that appear indistinguishable from a real pair.

As discussed above, we will aim to understand the hardness assumptions required to build *weak* variants of zero-knowledge with non-negligible security errors. Specifically, an $(\epsilon_{\mathsf{s}}, \epsilon_{\mathsf{zk}})$-weak NIZK allows a cheating prover to convince a verifier of the correctness of a *false* statement with probability at most ϵ_{s}, and allows an efficient verifier to distinguish a real proof from a simulated one with probability at most ϵ_{zk}. As also noted in prior works, these are trivial when $\epsilon_{\mathsf{s}} + \epsilon_{\mathsf{zk}} \geq 1$, since the CRS can always include an ϵ_{s}-biased bit indicating whether to use the trivially sound system where the witness is sent in the clear, or the trivially ZK system where the prover sends nothing, and the verifier accepts. Therefore, we are only concerned with the *non-trivial* setting of errors, i.e. $\epsilon_{\mathsf{s}} + \epsilon_{\mathsf{zk}} < 1$, as we investigate the following question:

Do $(\epsilon_{\mathsf{s}}, \epsilon_{\mathsf{zk}})$*-weak NIZK for* NP *with non-trivial* $(\epsilon_{\mathsf{s}}, \epsilon_{\mathsf{zk}})$ *imply the existence of one-way functions?*

Amplifying Weakly-secure NIZKs. Weakly-secure NIZKs for NP with non-negligible soundness and zero-knowledge errors have a rich history of study [GJS19, BKP+24, BG24]. Goyal, Jain and Sahai [GJS19] first investigated the problem of *amplifying* the security of such NIZKs, to ultimately reduce both the soundness and ZK errors to negligible. They

showed that $(\epsilon_{\mathsf{s}}, \epsilon_{\mathsf{zk}})$-weak NIZK for any constant $\epsilon_{\mathsf{s}} + \epsilon_{\mathsf{zk}} < 1$ can be generically amplified to obtain NIZKs with negligible security errors, *additionally assuming sub-exponentially-secure public-key encryption* (PKE). The subsequent work of Bitansky and Geier [BG24] showed that the public-key assumption can be relaxed to *only assume the existence of one-way functions*, for the setting of *statistically sound* NIZKs, and to only assume polynomially-secure PKE, for the setting of (computationally sound) NIZK arguments. This leaves open the following prominent question:

Can $(\epsilon_{\mathsf{s}}, \epsilon_{\mathsf{zk}})$-weak NIZK for NP with non-trivial $(\epsilon_{\mathsf{s}}, \epsilon_{\mathsf{zk}})$ be amplified unconditionally to obtain NIZK for NP with negligible errors?

Our Results. We proceed to informally discuss our results. In what follows, by "standard" NIZK we mean NIZK with negligible soundness and ZK errors. Our main theorem establishes that worst-case hardness of NP, together with the existence of $(\epsilon_{\mathsf{s}}, \epsilon_{\mathsf{zk}})$-weak NIZK arguments for NP with constant $\epsilon_{\mathsf{zk}} + \sqrt{\epsilon_{\mathsf{s}}} < 1$ implies the existence of one-way functions.

Theorem 1. *(Informal). As long as $\mathsf{NP} \not\subseteq \mathsf{ioP/poly}$, for any constants ϵ_{zk} and ϵ_{s} such that $\epsilon_{\mathsf{zk}} + \sqrt{\epsilon_{\mathsf{s}}} < 1$, the existence of $(\epsilon_{\mathsf{s}}, \epsilon_{\mathsf{zk}})$-weak NIZK* arguments *implies the existence of one-way functions.*

While the above theorem statement covers a wide range of NIZK errors (shaded in blue in Figure 1), there is a slim range of non-trivial parameters corresponding to $\epsilon_{\mathsf{zk}} + \sqrt{\epsilon_{\mathsf{s}}} \geq 1$, but $\epsilon_{\mathsf{zk}} + \epsilon_{\mathsf{s}} < 1$ for which we do not obtain an implication to one-way functions (this corresponds to the red shaded region in Figure 1). Understanding whether ZK proof systems with errors in this region imply one-way functions remains an intriguing open problem.

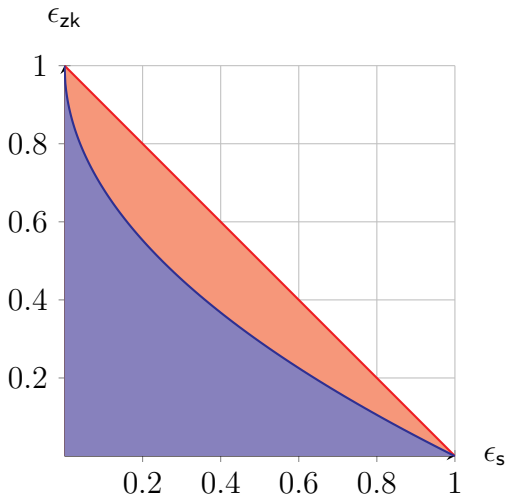

Figure 1: We shade values of $\epsilon_{\mathsf{s}}, \epsilon_{\mathsf{zk}}$ for which non-trivial $(\epsilon_{\mathsf{s}}, \epsilon_{\mathsf{zk}})$-weak NIZKs exist. The blue region represents parameters for which we obtain one-way functions from $(\epsilon_{\mathsf{s}}, \epsilon_{\mathsf{zk}})$-weak NIZKs as long as $\mathsf{NP} \not\subseteq \mathsf{ioP/poly}$. The red region corresponds to parameters for which non-trivial NIZKs may exist, but obtaining an implication to one-way functions remains open.

As a warm-up to our main result, we also show that the techniques from [OW93] can be adapted to get a simpler proof, albeit for a smaller parameter regime.[1]

Theorem 2. *(Informal). As long as* $\mathsf{NP} \not\subseteq \mathsf{ioP/poly}$*, for any* ϵ_{zk} *and* ϵ_{s} *such that* $\epsilon_{\mathsf{s}}+2\epsilon_{\mathsf{zk}} < 1$*, the existence of* $(\epsilon_{\mathsf{s}}, \epsilon_{\mathsf{zk}})$*-weak NIZK* arguments *implies the existence of one-way functions.*

Amplification. Next, we observe that weak NIZKs for a wide range of error parameters are known to imply (standard) NIZKs as long as one-way functions exist [BG24]. Our theorem above can be viewed as eliminating the need to make an extra assumption about the existence of one-way functions when amplifying NIZK. Specifically, if NP is easy in the worst case then NIZKs for NP exist unconditionally, whereas if NP is hard in the worst case, then our theorem above builds one-way functions from weak NIZKs for NP. The resulting one-way functions can be used, together with weak NIZKs, to obtain standard NIZKs by relying on known amplification theorems in [BG24]. We summarize these amplification results below. For technical reasons that we discuss below, we will need to make the extremely mild complexity-theoretic assumption (Assumption 1) that *if* $\mathsf{NP} \subseteq \mathsf{ioP/poly}$*, then* $\mathsf{NP} \subseteq \mathsf{BPP}$.

Theorem 3. *(Informal). Under Assumption 1, for any constants* ϵ_{zk} *and* ϵ_{s} *such that* $\epsilon_{\mathsf{zk}} + \sqrt{\epsilon_{\mathsf{s}}} < 1$*, the existence of* $(\epsilon_{\mathsf{s}}, \epsilon_{\mathsf{zk}})$*-weak NIZK* proofs *implies the existence of standard NIZK proofs.*

Theorem 4. *(Informal). Under Assumption 1, for any negligible* ϵ_{s} *and constant* $\epsilon_{\mathsf{zk}} < 1$*, the existence of* $(\epsilon_{\mathsf{s}}, \epsilon_{\mathsf{zk}})$*-weak NIZK* arguments *implies the existence of (standard) NIZK arguments with negligible soundness and ZK errors.*

We note that the only remaining obstacle towards unconditionally establishing that weak NIZKs with parameters above imply standard NIZK is to prove Assumption 1, namely that if NP languages are efficiently decidable infinitely often with advice, then they are efficiently decidable on all input lengths. We find this to be an extremely mild complexity assumption (indeed, it is implied by $\mathsf{NP} \not\subseteq \mathsf{ioP/poly}$, which is already a mild worst case assumption). Assuming that one can bridge the gap between infinitely-often and everywhere hardness appears to be somewhat necessary approaches like ours that rely on an intermediate primitive such as one-way functions.

Open Problems. Our work leaves open many fascinating questions.

1. The most obvious open problem is to close the gaps left open by this work and understand if we can *unconditionally* amplify weak NIZKs with *all* non-trivial error parameters.

2. It is also interesting to study whether our techniques will extend to amplifying *interactive* zero-knowledge proof systems: it appears that a simple extension of the Ostrovsky-Wigderson [OW93] techniques, which this paper builds on, will only enable us to amplify k-round interactive protocols with $\epsilon_{\mathsf{s}} + k\epsilon_{\mathsf{zk}} < 1$. It may be interesting

[1] As stated here, Theorem 2 is strictly weaker than Theorem 1. However, we note that in our formal statement we allow the completeness error to also be a parameter, which creates some limited parameter regimes where Theorem 2 applies but Theorem 1 does not.

to see how much the dependence on k can be removed; our work can be viewed as removing this dependence for the setting of $k = 2$.

3. Finally, an important open problem is to understand if *stronger* cryptographic primitives, such as public-key encryption are implied by (standard or weak) NIZKs.

2 Technical Overview

2.1 The Ostrovsky-Wigderson [OW93] Analysis

We start by revisiting the analysis of [OW93], who showed that ZK proofs with negligible errors, together with average-case hardness of NP, imply one-way functions. We will see where their analysis fails when applied to NIZK proofs with large soundness and zero-knowledge errors.

Let $(\mathrm{Gen}, \mathrm{P}, \mathrm{V})$ be an $(\epsilon_{\mathsf{s}}, \epsilon_{\mathsf{zk}})$-weak NIZK for some language $\mathcal{L}$. The key tool that [OW93] uses is universal extrapolation (UE): roughly, given the description of a polynomial-time sampler Samp and a sample y from its output distribution, it is possible to efficiently sample uniformly (up to arbitrarily small inverse-polynomial error) from the distribution of all inputs on which Samp outputs y. Informally, UE allows us to "reverse sample" the randomness used to generate a given CRS, getting (approximately) a random r such that $\mathrm{Sim}(x; r)$ would output the same crs. From [IL90], we know that UE is possible as long as one-way functions do not exist.

Using this tool (which requires assuming the non-existence of one-way functions), consider the following algorithm $\mathcal{A}$ to decide if $x \in \mathcal{L}$: sample $\mathsf{crs} \leftarrow \mathrm{Gen}(1^{|x|})$, use UE to "reverse sample" an r such that $\mathrm{Sim}(x; r)$ would output the same CRS, and accept if and only if the honest verifier would accept the proof output by $\mathrm{Sim}(x; r)$. We will now inspect the behavior of $\mathcal{A}$ on instances within and outside the language.

- First, consider the case where $x \notin \mathcal{L}$. We note that $\mathcal{A}$ will only accept if it finds an accepting proof for x relative to an honestly-generated CRS. Since $\mathcal{A}$ is efficient, soundness tells us that this occurs with probability at most ϵ_{s}.

- Next, consider the case where $x \in \mathcal{L}$. First, note that if we generated an honest CRS and proof, they would be accepted with probability 1.[2] By zero-knowledge, if we instead generated the CRS and proof from Sim, these must be accepted with probability at least $1 - \epsilon_{\mathsf{zk}}$.

  Now consider an algorithm $\mathcal{B}$ that is identical to $\mathcal{A}$, except that it starts by generating a CRS from Sim (ignoring the proof) instead of from Gen. By the guarantees of UE, we know that the distribution of the CRS and proof that $\mathcal{B}$ runs verification on is close (up to an arbitrary inverse-polynomial error) to the distribution output by Sim. Combining this with the above (and ignoring the arbitrarily small error term for simplicity), we have that $\mathcal{B}$ must accept with probability at least $1 - \epsilon_{\mathsf{zk}}$.

[2] For the purposes of the introduction, we are ignoring completeness error; our formal analysis will have an additional parameter for ϵ_{c}.

Finally, we note that because the only difference between $\mathcal{A}$ and $\mathcal{B}$ is whether they start with a real or simulated CRS, their output distributions must be ϵ_{zk}-close to one another. Thus in particular, $\mathcal{A}$ must accept $x \in \mathcal{L}$ with probability at least $1 - 2\epsilon_{\mathsf{zk}}$.

Overall, we get that for infinitely many n, $\mathcal{A}$ accepts $x \in \mathcal{L}$ with probability at least $1 - 2\epsilon_{\mathsf{zk}}$ and accepts $x \notin \mathcal{L}$ with probability at most ϵ_{s}. Whenever $1 - 2\epsilon_{\mathsf{zk}}$ is noticeably larger than ϵ_{s}, this gives an efficient algorithm to decide $\mathcal{L}$. That is, if one-way functions don't exist, only efficiently decidable languages have NIZKs with $\epsilon_{\mathsf{s}} + 2\epsilon_{\mathsf{zk}}$ noticeably smaller than 1.

Unfortunately, this analysis doesn't allow us to say anything about protocols for which $\epsilon_{\mathsf{zk}} \geq \frac{1}{2}$, much less any that have $\epsilon_{\mathsf{zk}} \approx 1$. In particular, we note that this analysis applies the zero-knowledge property twice, and so has to pay double the zero-knowledge error.

2.2 Tightening the Analysis

Our first technical goal is to also obtain meaningful implications to one-way functions from NIZK proofs with $\epsilon_{\mathsf{zk}} \geq \frac{1}{2}$. In other words, we would like to avoid paying for the ZK error twice in our analysis.

If we take a careful look at the change from $\mathcal{B}$ to $\mathcal{A}$ in the above analysis, we notice that we use the zero-knowledge guarantee in this case to *only change the CRS* from simulated to real (ignoring the proof). This gives us an opening to tighten our analysis.

Towards building some intuition, we will first assume that the soundness error is small (it may help to assume that ϵ_{s} is 0, but this is not strictly necessary for our analysis). We will carefully inspect the probability that the machine $\mathcal{B}$ described above outputs an accepting proof on a real versus simulated CRS, given $x \notin \mathcal{L}$.[3] Fix some such x. We will call a crs *bad* if given crs and x, $\mathcal{B}$ outputs an accepting proof with high probability. By soundness, we know that the real CRS distribution samples a *bad* CRS with low probability. On the other hand, the only way for $\mathcal{B}$ to output accepting proofs given simulated CRS is if the simulated CRS distribution has a large probability of sampling a *bad* CRS. Overall, this suggests that the error ϵ_{zk} incurred in the analysis of this case can be reduced by having the algorithm discard any *bad* CRSes.

Formalizing this, we consider the following candidate decider $\mathcal{A}$ for $\mathcal{L}$: given x, we start by sampling a CRS and proof from $\mathrm{Sim}(x)$. If the CRS we get is m times more likely to occur when sampled from the simulated distribution as compared to the real distribution (where m is a value to be set later),[4] $\mathcal{A}$ will immediately reject x. Otherwise, $\mathcal{A}$ accepts if and only if the honest verifier would accept the sampled CRS and proof.

We first consider what $\mathcal{A}$ will do on $x \in \mathcal{L}$. In this case, we know that the output distribution of $\mathcal{A}$ can change by at most ϵ_{zk} if we replace the simulated CRS and proof with honestly generated ones. But now we claim that with this modification, $\mathcal{A}$ can reject $x \in \mathcal{L}$

[3] In the above analysis, note that this step was actually in the case where $x \in \mathcal{L}$. However, for our improved analysis, it will turn out to be easier to change which hybrid is our candidate algorithm so that we can directly use the soundness error in this part; see more details below.

[4] Efficiently deciding whether a given CRS is m times more likely to occur when sampled from the simulated distribution as compared to the real distribution will turn out to be fairly involved. We will discuss how this is done in the next section.

with probability at most $\frac{1}{m}$. Indeed, the probability that the real CRS distribution samples a string that is m times more likely to have come from the simulated distribution (or in fact *any* other fixed distribution) is at most $\frac{1}{m}$. Thus, the probability that $\mathcal{A}$ outputs an accepting proof for $x \in \mathcal{L}$ can't drop below $1 - \epsilon_{\mathsf{zk}} - \frac{1}{m}$.

The more interesting question is what happens when $x \notin \mathcal{L}$. Here, recall that we built a machine $\mathcal{B}$ that given any (crs, x) relies on universal extrapolation to reverse-sample r, and then generate a simulated proof for x. This machine will also be modified to discard any crs that is m times more likely to have come from the simulated (as opposed to real) distribution. At this point, we previously applied the zero-knowledge property to switch between a real and simulated CRS, thereby incurring an extra ϵ_{zk} overhead. Our improved analysis will instead consider for each possible CRS the probability that $\mathcal{B}$ generates an accepting transcript when given that particular CRS. Now note that every CRS must either (1) be at most m times more likely to come from the simulated distribution instead of the real one, or (2) be guaranteed to be rejected. Thus, the probability of accepting if we start from a simulated CRS is at most m times the probability of accepting if we start from a real CRS. Since the latter is at most ϵ_{s}, we have that (ignoring arbitrarily small inverse-polynomial terms), the probability of our algorithm accepting $x \notin \mathcal{L}$ is at most $m\epsilon_{\mathsf{s}}$.

Putting this all together, our updated algorithm $\mathcal{A}$ (infinitely-often) accepts $x \in \mathcal{L}$ with probability at least $1 - \epsilon_{\mathsf{zk}} - \frac{1}{m}$ and $x \notin \mathcal{L}$ with probability at most $m\epsilon_{\mathsf{s}}$. In order for this to be interesting, we need these two quantities to have a noticeable gap, or equivalently to have $\epsilon_{\mathsf{zk}} + \frac{1}{m} + m\epsilon_{\mathsf{s}}$ noticeably smaller than 1. Setting $m = \frac{1}{\sqrt{\epsilon_{\mathsf{s}}}}$ will minimize this quantity, telling us that if one-way functions don't exist, only languages that can be decided efficiently (infinitely often) can have NIZKs with $\epsilon_{\mathsf{zk}} + 2\sqrt{\epsilon_{\mathsf{s}}}$ noticeably smaller than 1.

As one final optimization, we note that NIZK amplification theorems from [BG24] can show that if we start from a protocol with $\epsilon_{\mathsf{zk}} + \sqrt{\epsilon_{\mathsf{s}}}$ noticeably smaller than 1, an appropriate number of parallel repetitions will give us a protocol with $\epsilon_{\mathsf{zk}} + 2\sqrt{\epsilon_{\mathsf{s}}}$ noticeably smaller than 1. Thus, if one-way functions don't exist, only languages that can be efficiently decided (infinitely often) can have NIZKs with $\epsilon_{\mathsf{zk}} + \sqrt{\epsilon_{\mathsf{s}}}$ noticeably less than 1.

2.3 Using Universal Approximation

The previous section glossed over a rather important detail: how do we efficiently determine if a CRS is much more likely to be simulated than real? A natural candidate for this is to use Universal Approximation (UA), which allows us to (approximately) compute the probability of a particular output being drawn from a given efficiently-samplable distribution. UA is known to be possible as long as one-way functions don't exist [IL90]. Unfortunately, this isn't quite enough for our purposes: the correctness of UA only guarantees that it is correct with high probability *over the outputs of the distribution we are working with*, whereas our algorithm $\mathcal{A}$ needs to accurately compute the probability of a CRS being drawn from both the real and simulated distributions, regardless of how that CRS was actually sampled.

To overcome this issue, we introduce a strengthening of UA, which we term one-sided error UA (1UA). In addition to the standard requirement that the universal approximator is (approximately) correct with high probability over an output from the distribution $\mathcal{D}$ it is trying to estimate, we also require that *for every* string y, regardless of how likely the string is to occur in the output of $\mathcal{D}$, the approximator will not *over*estimate the probability assigned

by $\mathcal{D}$ to y. Indeed, this is sufficient: with 1UA, in the $x \notin \mathcal{L}$ case the only remaining error is *under*estimating the probability of the CRS coming from the real distribution (which can only make the algorithm more likely to reject), while in the $x \in \mathcal{L}$ case the only remaining error is *under*estimating the probability of the CRS coming from the simulated distribution (which can only make the algorithm more likely to accept).

2.4 Constructing 1UA

We now need to show that 1UA exists as long as one-way functions do not. This will be the most technically involved part of our work.

We will begin by first giving a brief overview of how to construct UA assuming one-way functions do not exist, as was originally shown in in [IL90, IL89, Imp92]. Recall that a universal approximator is given a value y and needs to estimate the probability of y being drawn from some efficiently-sampleable distribution $\mathcal{D}$. For our proofs, we will require this estimate to be correct up to multiplicative $(1 \pm 1/\mathsf{poly})$ errors. That is, for every polynomial $q(\cdot)$ and every efficiently sampleable distribution $\mathcal{D}$, the universal approximator (parameterized by $q, \mathcal{D}$) given some y must (whp) output a guess $\widetilde{p}_y$ for $p_y = \Pr_{\mathcal{D}}[y]$ satisfying:

$$\Pr_{y \leftarrow \mathcal{D}}\left[p_y\left(1 - \frac{1}{q(n)}\right) \leq \widetilde{p}_y \leq p_y\left(1 + \frac{1}{q(n)}\right)\right] \geq 1 - \frac{1}{q(n)} \tag{1}$$

As a first step towards building such UA, note that prior work [Imp92] implicitly builds a universal approximator that can make much *larger* (polynomial, multiplicative) errors in approximating p_y, and outputs a guess $\widehat{p}_y$ satisfying:

$$\Pr_{y \leftarrow \mathcal{D}}[p_y \leq \widehat{p}_y \leq p_y(n^2 q(n))] \geq 1 - \frac{1}{q(n)} \tag{2}$$

We will call this approximation $\widehat{p}_y$ that we get from [Imp92] a "coarse estimate" of the probability of sampling y from $\mathcal{D}$, since it may be off from the true value by a factor larger than what our eventual analysis requires.

Given this coarse estimate $\widehat{p}_y$, we will now describe how to get a more precise estimate. We can first convert $\widehat{p}_y$ to an estimate of the *number* of random strings r that would cause $\mathcal{D}$ to output y, call this $\widehat{k}_y$ which will simply equal $\widehat{p}_y \cdot 2^{|r|}$. Next, define an efficient function f such that $f(r) = (\mathcal{D}(r), h(r))$, where h is a pairwise independent hash whose output space is chosen to be (sufficiently) polynomially larger than $\widehat{k}_y$. Since (weak) one-way functions don't exist, we know there exists an efficient f^{-1} that inverts f with high probability over the initial sampling of r. To get an finer estimate $\widetilde{k}_y$ of the number of random strings that lead $\mathcal{D}$ to output y, we run f^{-1} a polynomial number of times on input y and random points in the output space of our hash. Our estimate of the number of random strings is then the proportion of these runs that succeed times the size of the hash space; we can convert this to an estimate of the probability as $\widetilde{p}_y = \widetilde{k}_y \cdot 2^{-|r|}$.

To see why this updated algorithm satisfies our definition of UA with low multiplicative errors (equation 1), consider what happens over a random y sampled from $\mathcal{D}$. Let k_y denote the actual number of preimages of y (under $\mathcal{D}$). By the guarantees of the coarse estimator, with high probability $\widehat{k}_y \geq k_y$, so the output space of the hash function will be polynomially

larger than k_y. By pairwise independence, with high probability over a random preimage r of y, no other preimage r' of y will have $h(r) = h(r')$. Thus, we can get a good estimate of the number of preimages by instead estimating how many distinct values they hash to. Since we know the size of the output space of h, it suffices to estimate the probability that a random z in that space has an r such that $f(r) = (y, z)$. To show that we get a good estimate of this, we now use the fact that (with high probability) $\widehat{k}_y$ is only polynomially larger than k_y. This means that the true probability of such an r existing is at least some inverse polynomial value. Since we run f^{-1} a (sufficiently large) polynomial number of times on random z, a Chernoff bound will tell us that we in fact get a very precise estimate of the desired probability.

Finally (and most importantly), we will show that this algorithm also satisfies 1UA, as opposed to just UA. This means that we must additionally show that for *every* y, the guess $\widetilde{p}_y$ output by our algorithm is not an overestimate, i.e. it satisfies

$$\Pr\left[\widetilde{p}_y \leq p_y\left(1 - \frac{1}{q(n)}\right)\right] \geq 1 - \frac{1}{q(n)} \tag{3}$$

where the probability is over the coins of the 1UA algorithm. For this analysis, we will first pessimistically assume that no two r such that $\mathcal{D}(r) = y$ collide under h, as this can only increase the number of z such that $f(r) = (y, z)$, which in turn can only increase the odds of our algorithm overestimating. Next, a closer inspection of the proof in [Imp92] reveals that the coarse approximator also rarely overestimates, even on values of y that are unlikely to come from $\mathcal{D}$. Specifically, for *every* y, the coarse-grained estimate satisfies

$$\Pr\left[\widehat{p}_y \leq p_y\left(n^2 q(n)\right)\right] \geq 1 - \frac{1}{q(n)} \tag{4}$$

This equation implies that the coarse estimate $\widehat{k}_y$ is (with high probability) at most polynomially larger than the actual k_y. This, combined with our (pessimistic) assumption that no two distinct r, r' have $f(r) = f(r') = (y, z)$ for any z, implies that the probability that a random z has an inverse under f is an inverse polynomial. Thus, since we run f^{-1} a (sufficiently large) polynomial number of times, a Chernoff bound will tell us that we get a very precise estimate of the probability that a random z has an inverse under f, and so in particular are unlikely to overestimate that probability, regardless of the value of y. This completes an overview of our 1-universal approximator.

2.5 Relying only on Worst-Case Hardness

Our techniques described above still don't quite give us the desired results. In particular, to argue that any language with a weak NIZK is efficiently decidable, we need to show that (for infinitely many instance sizes) the algorithm we give works for *every* instance. This means that when we use UE and 1UA, they need to be given the instance as an auxiliary input, and so their constructions will rely on the non-existence of *auxiliary-input* one-way functions. Conversely, if we want to base our results on the non-existence of standard one-way functions, the UE and 1UA constructions we use can only be given the instance size, and so will only be guaranteed to work on average over a distribution of instances of that

size. In particular, this will mean that our decider $\mathcal{A}$ will only work in the average case, and so we will only contradict the average-case (as opposed to worst-case) hardness of NP.

In effect, this means that we would get two incomparable results:

1. A NIZK for any language that is hard in the worst case implies the existence of auxiliary-input one-way functions, and

2. A NIZK for any language that is hard on average implies the existence of standard one-way functions.

Fortunately, [HN24] recently showed how to bridge this gap for the original proof from [OW93], and with some work, we show that their techniques can be adapted to our setting.

In fact, we closely follow the simplified proof of [HN24] given in [LMP24], where the key step is to show that the existence of an auxiliary-input one-way function implies the existence of a language $\mathcal{L}$ in NP that is hard-on-average against errorless polynomial-time adversaries. In more detail, they consider a class of admissible adversaries that are allowed to output $\perp$ (indicating that they cannot decide the instance) on some fraction of randomly sampled instances. They show that every such (efficient, admissible) adversary will output an incorrect answer (with sufficiently high probability) on some instance in $\mathcal{L}$.

We can now hope to obtain one-way functions in two steps (following ideas in [HN24, LMP24]):

- Step 1: Combining the above paragraph with result (1) above, we get that a NIZK for any language that is hard in the worst case implies the existence of a language $\mathcal{L}$ in NP that is hard-on-average against errorless polynomial-time adversaries.

- Step 2: Next, we hope to able to rely on the existence of NIZKs for this particular hard-on-average language to combine with result (2) above, and obtain standard one-way functions.

Unfortunately, a key problem with executing Step 2 is that the type of "errorless" average-case hardness obtained as a result from Step 1 is insufficient to instantiate result (2). However, we show that it is possible to *strengthen* the analysis of [HN24, LMP24] to also show that auxiliary-input one-way functions imply a *stronger* form of average-case hardness, against a larger class of adversaries. Specifically, these adversaries are required to be correct with high probability on *every* $x \in \mathcal{L}$, but are allowed to be correct only *on average* over random $x \notin \mathcal{L}$ (a precise definition appears in Section 3.6). With this strengthened definition of average-case hardness, we are also able to execute Step 2, thereby completing our proof.

2.6 Applications to NIZK Amplification

As an immediate application of our work, we note that we can combine it with recent results from [BG24] to show that NIZKs for NP amplify (almost) unconditionally. In particular, [BG24] showed that as long as one-way functions exist, (1) NIZKs with *statistical* soundness can be amplified to have negligible errors as long as $\epsilon_{\mathsf{s}} + \epsilon_{\mathsf{zk}} < 1$, and (2) NIZKs with *computational* soundness can be amplified to have negligible errors as long as ϵ_{s} is negligible

and $\epsilon_{\mathsf{zk}} < 1$. (We note as a technicality that they also require ϵ_{s} and ϵ_{zk} to be constants for the first result, and ϵ_{zk} to be a constant for the second.)

Since our results say that high-error NIZKs for hard languages in NP already imply one-way functions on their own, we can replace the assumption of one-way functions with simply the worst-case hardness of NP. In particular, we get that as long as $\mathsf{NP} \not\subseteq \mathsf{ioP/poly}$[5],

1. NIZKs for NP-hard languages with *statistical* soundness can be amplified to have negligible errors as long as $\epsilon_{\mathsf{zk}} + \sqrt{\epsilon_{\mathsf{s}}} < 1$,[6] and
2. NIZKs for NP-hard languages with *computational* soundness can be amplified to have negligible errors as long as $\epsilon_{\mathsf{zk}} < 1$.

In fact, we note that the two statements above are trivially true if $\mathsf{NP} \subseteq \mathsf{BPP}$, as in that case every language in NP has a trivial NIZK. Thus, our amplification results are *almost* unconditional: to amplify, we only need to assume that if $\mathsf{NP} \subseteq \mathsf{ioP/poly}$ then $\mathsf{NP} \subseteq \mathsf{BPP}$. We end up having to make this (rather mild) assumption because of the cryptographic nature of the assumption that one-way functions do not exist, which only gives us a non-uniform machine that succeeds infinitely-often at certain algorithmic tasks. This completes our overview.

3 Preliminaries

3.1 Notation

We define the following notation:

- $x[:i]$ represents the first i bits of a string x.
- $[n]$ represents the set $\{1, 2, \ldots, n\}$.
- We call a function μ negligible if it is smaller than any inverse polynomial, that is if for every polynomial p there exists an $n_0 \in \mathbb{N}$ such that for all $n \geq n_0$, $\mu(n) < \frac{1}{p(n)}$. We let negl denote an unspecified negligible function.
- We call a function μ noticeable if it is asymptotically larger than some inverse polynomial, that is if there exists a polynomial p and $n_0 \in \mathbb{N}$ such that for all $n \geq n_0$, $\mu(n) \geq \frac{1}{p(n)}$.
- $a <_n b$ represents that a is noticeably less than b, that is that there exists a polynomial p and $n_0 \in \mathbb{N}$ such that for all $n \geq n_0$, $a(n) < b(n) - \frac{1}{p(n)}$.

[5] If (non-uniformly secure) one-way functions do not exist, our reduction/decider for the language described in previous sections will only be able to place $\mathcal{L} \in \mathsf{ioP/poly}$ instead of BPP. The infinitely-often constraint is needed because a one-way function is considered "broken" if we can invert it on infinitely many input sizes, thus our universal extrapolation will only work infinitely often. Additionally, determining if a CRS looks "too simulated" by running UA will depend on the ability to break a one-way function. Since the standard definition of one-way functions (including the definition required by amplification theorems in [BG24]) requires security against non-uniform adversaries, one-way functions not existing only guarantees us a non-uniform algorithm to break any given candidate.

[6] Note that there is a gap here, since the results of [BG24] can work for protocols where $\epsilon_{\mathsf{zk}} + \epsilon_{\mathsf{s}} < 1 < \epsilon_{\mathsf{zk}} + \sqrt{\epsilon_{\mathsf{s}}}$, but ours do not. We leave it to future work to determine if this gap can be closed.

3.2 Uniform and Non-Uniform Machines

Throughout this paper, our protocols will have (honest) participants that can be described as uniform Turing machines, whereas adversaries will (w.l.o.g.) be non-uniform Turing Machines, or equivalently, families of polynomial-sized circuits.

3.3 One-Way Functions

Definition 1 (One-way Functions)**.** *Fix an alphabet Σ. We say that a family of functions $\{f_x\}_{x\in\Sigma^*}$ with $f_x : \{0,1\}^{m_i(|x|)} \to \{0,1\}^{m_o(|x|)}$ is one-way if*

- ***[Efficiency]*** *There exists a Turing machine M and polynomial p such that for all $x \in \Sigma^*$ and $r \in \{0,1\}^{m_i(|x|)}$, $M(x,r)$ outputs $f_x(r)$ in time at most $p(|x|)$*
- ***[One-Wayness]*** *For all non-uniform PPT machines $\mathcal{A}$, all polynomials p, and all sufficiently large n, there exists an $x \in \Sigma^n$ with*

$$\Pr\Big[f_x(\mathcal{A}(x, f_x(r))) = f_x(r) | r \xleftarrow{\$} \{0,1\}^{m_i(|x|)}\Big] \leq \frac{1}{p(n)}$$

Definition 2 (Weak One-way Functions)**.** *Fix an alphabet Σ. We say that a family of functions $\{f_x\}_{x\in\Sigma^*}$ with $f_x : \{0,1\}^{m_i(|x|)} \to \{0,1\}^{m_o(|x|)}$ is weakly one-way if*

- ***[Efficiency]*** *There exists a Turing machine M and polynomial p such that for all $x \in \Sigma^*$ and $r \in \{0,1\}^{m_i(|x|)}$, $M(x,r)$ outputs $f_x(r)$ in time at most $p(|x|)$*
- ***[Weak One-Wayness]*** *For all non-uniform PPT machines $\mathcal{A}$ there exists a polynomial p such that for sufficiently large n there exists an $x \in \Sigma^n$ with*

$$\Pr\Big[f_x(\mathcal{A}(x, f_x(r))) = f_x(r) | r \xleftarrow{\$} \{0,1\}^{m_i(|x|)}\Big] \leq 1 - \frac{1}{p(n)}$$

Remark 1. *A few remarks about these definitions are in order:*

- *The definition above can be used to define both (standard) one-way functions, and one-way functions with auxiliary input. If we take $\Sigma = \{0,1\}$, these definitions correspond to (weak) one-way functions with auxiliary inputs; if we take $\Sigma = \{1\}$, these definitions correspond to (weak) one-way functions with no auxiliary input. In this paper, if we refer to an "auxiliary input (weak) one-way function", we mean one with respect to $\Sigma = \{0,1\}$; if we don't otherwise specify Σ, we default to $\Sigma = \{1\}$. A similar remark applies to all our definitions that make reference to Σ.*
- *Since we require that M is computable in time polynomial in $|x|$ (as opposed to in the size of its full input), we can assume WLOG that m_i and m_o are polynomially-bounded functions.*

While weak one-way functions may at first seem much weaker than one-way functions, the two notions are existentially equivalent.

Theorem 5 ([Yao82])**.** *Fix an alphabet Σ. One-way functions exist with respect to Σ if and only if weak one-way functions do.*

3.4 Universal Extrapolation

Definition 3 (Sampler)**.** *Fix an alphabet Σ. We say that a probabilistic polynomial time machine M is a sampler from m_i input bits to m_o output bits if for all $x \in \Sigma^*$, $M(x)$ uses at most $m_i(|x|)$ bits of randomness and for all $r \in \{0,1\}^{m_i(|x|)}$, $M(x;r) \in \{0,1\}^{m_o(|x|)}$.*

Remark 2. *Note that m_i and m_o can WLOG be bounded by the run time of M; since M runs in time polynomial in $|x|$, we can freely assume that m_i and m_o are polynomially-bounded functions of n.*

Definition 4 (Infinitely-Often Universal Extrapolation)**.** *Fix an alphabet Σ. We say that Infinitely-Often Universal Extrapolation (io-UE) is possible if for every sampler M from m_i input bits to m_o output bits and all polynomials p, there exists a non-uniform PPT machine N such that for infinitely many n, the following two distributions are $\frac{1}{p(n)}$-close in statistical distance for every $x \in \Sigma^n$:*

1. *Sample $r \xleftarrow{\$} \{0,1\}^{m_i(n)}$ and output $r \| M(x;r)$*
2. *Sample $r \xleftarrow{\$} \{0,1\}^{m_i(n)}$ and output $N(x, M(x;r)) \| M(x;r)$*

Theorem 6 ([IL90])**.** *Fix an alphabet Σ. io-UE is possible with respect to Σ if and only if one-way functions do* not *exist with respect to Σ.*

3.5 Non-Interactive Zero-Knowledge

Definition 5 ($(\epsilon_{\mathsf{c}}, \epsilon_{\mathsf{s}}, \epsilon_{\mathsf{zk}})$-NIZK)**.** *An $(\epsilon_{\mathsf{c}}, \epsilon_{\mathsf{s}}, \epsilon_{\mathsf{zk}})$-NIZK for an NP relation $\mathcal{R}$ is a tuple of polynomial-time algorithms* $(\mathrm{Gen}, \mathrm{P}, \mathrm{V})$ *such that*

- ***Syntax***
  - $\mathsf{crs} \leftarrow \mathrm{Gen}(1^n)$*:* Gen *is a randomized algorithm that on input an instance size (in unary) outputs a common reference string.*
  - $\pi \leftarrow \mathrm{P}(\mathsf{crs}, x, w)$*:* P *is a randomized algorithm that on input a crs, instance, and witness for that instance outputs a proof.*
  - $b \leftarrow \mathrm{V}(\mathsf{crs}, x, \pi)$*:* V *is a* deterministic *algorithm that on input a crs, instance, and proof for that instance outputs a bit denoting acceptance or rejection.*
- ***Completeness****: For every $(x, w) \in \mathcal{R}$,*

$$\Pr\left[\mathrm{V}(\mathsf{crs}, x, \pi) = 1 \middle| \begin{matrix} \mathsf{crs} \leftarrow \mathrm{Gen}(1^{|x|}) \\ \pi \leftarrow \mathrm{P}(\mathsf{crs}, x, w) \end{matrix}\right] \geq 1 - \epsilon_{\mathsf{c}}(|x|)$$

- ***Soundness****: For any* non-uniform *PPT algorithm $\tilde{\mathrm{P}}$, we have that for $x \notin \mathcal{L}_{\mathcal{R}}$ such that $|x|$ is large enough,*

$$\Pr\left[\mathrm{V}(\mathsf{crs}, x, \tilde{\pi}) = 1 \middle| \begin{matrix} \mathsf{crs} \leftarrow \mathrm{Gen}(1^{|x|}) \\ \tilde{\pi} \leftarrow \tilde{\mathrm{P}}(\mathsf{crs}, x) \end{matrix}\right] \leq \epsilon_{\mathsf{s}}(|x|)$$

- ***Zero-Knowledge***: *There exists a PPT simulator* Sim *such that for all non-uniform PPT distinguishers* $\mathcal{D}$ *and all* $(x, w) \in \mathcal{R}$ *with* $|x|$ *large enough,*

$$\left|\Pr\left[\mathcal{D}(\mathsf{crs}, \pi) = 1 \mid (\mathsf{crs}, \pi) \leftarrow \mathrm{Sim}(x)\right] - \Pr\left[\mathcal{D}(\mathsf{crs}, \pi) = 1 \middle| \begin{matrix} \mathsf{crs} \leftarrow \mathrm{Gen}(1^{|x|}) \\ \pi \leftarrow \mathrm{P}(\mathsf{crs}, x, w) \end{matrix}\right]\right| \leq \epsilon_{\mathsf{zk}}(|x|)$$

Remark 3. *We will often refer to a NIZK as being for the language defined by the relation* $\mathcal{R}$*, rather than for the relation itself.*

Remark 4. *We have defined soundness for NIZKs as being* computational *and* non-adaptive*. One can also consider* statistical *soundness, where our guarantee needs to hold even against inefficient* $\tilde{\mathrm{P}}$*. Additionally, one can consider* adaptive *soundness, where the adversary is allowed to choose the instance after seeing the CRS. Formally, our soundness requirement would be replaced by*

$$\Pr\left[\begin{matrix} |x| = n \wedge x \notin \mathcal{L}_{\mathcal{R}} \\ \wedge\ \mathrm{V}(\mathsf{crs}, x, \tilde{\pi}) = 1 \end{matrix} \middle| \begin{matrix} \mathsf{crs} \leftarrow \mathrm{Gen}(1^n) \\ (x, \tilde{\pi}) \leftarrow \tilde{\mathrm{P}}(\mathsf{crs}) \end{matrix}\right] \leq \epsilon_{\mathsf{s}}(n)$$

for sufficiently large n*.*

We make use of the following theorem on parallel repetition of NIZKs, closely adapted from [BG24].

Theorem 7 (Adapted from [BG24] Theorem 3.9)**.** *Let* $(\mathrm{Gen}, \mathrm{P}, \mathrm{V})$ *be an* $(\epsilon_{\mathsf{c}}, \epsilon_{\mathsf{s}}, \epsilon_{\mathsf{zk}})$*-NIZK for some language* $\mathcal{L}$*. Then if we take* $(\mathrm{Gen}', \mathrm{P}', \mathrm{V}')$ *as the* k*-fold repetition of* $(\mathrm{Gen}, \mathrm{P}, \mathrm{V})$*, there exist negligible functions* μ_{s} *and* μ_{zk} *such that* $(\mathrm{Gen}', \mathrm{P}', \mathrm{V}')$ *has soundness error at most* $\epsilon_{\mathsf{s}}^k + \mu_{\mathsf{s}}$ *and zero-knowledge error at most* $1 - (1 - \epsilon_{\mathsf{zk}})^k + \mu_{\mathsf{zk}}$*.*

We note that the original theorem in [BG24] considered an alternative definition of soundness, which they called soundness*. However, their proof still goes through with the standard definition as long as one considers *non-adaptive* soundness (as we do in this paper). For completeness, we have included a formal proof of this in Appendix A.

3.6 Complexity Classes

Definition 6 (Infinitely-Often BPP)**.** *Let* ioBPP *be the class of languages* $\mathcal{L}$ *such that there exists a PPT algorithm* $\mathcal{A}$*, functions* $a, b : \mathbb{N} \to [0, 1]$*, and a polynomial* p *such that for infinitely many* n*,*

- *For all* $x \in \mathcal{L} \cap \{0,1\}^n$*,* $\Pr[\mathcal{A}(x) = 1] \geq a(n)$
- *For all* $x \in \{0,1\}^n - \mathcal{L}$*,* $\Pr[\mathcal{A}(x) = 1] \leq b(n)$
- $a(n) - b(n) \geq \frac{1}{p(n)}$

We let $\mathsf{ioBPP/poly}$ *be defined similarly, except that* $\mathcal{A}$ *is allowed to be non-uniform.*

Definition 7 (Infinitely-Often P/poly)**.** *Let* $\mathsf{ioP/poly}$ *be the class of languages* $\mathcal{L}$ *such that there exists a non-uniform polynomial-time algorithm* $\mathcal{A}$ *such that for infinitely many* n*,*

- *For all* $x \in \mathcal{L} \cap \{0,1\}^n$, $\mathcal{A}(x) = 1$
- *For all* $x \in \{0,1\}^n - \mathcal{L}$, $\mathcal{A}(x) = 0$

Remark 5. *The textbook proof that* $\mathsf{BPP} \subseteq \mathsf{P/poly}$ *also says that* $\mathsf{ioBPP/poly} \subseteq \mathsf{ioP/poly}$*, so in particular* $\mathsf{ioBPP/poly} = \mathsf{ioP/poly}$*. Since* $\mathsf{ioP/poly}$ *is a more standard complexity class, we will state our results in terms of it, even when the algorithms we give are in fact probabilistic.*

Definition 8 (Average-Case Problem)**.** *We say that* $(\mathcal{L}, \mathcal{D})$ *is an average-case problem if* $\mathcal{L}$ *is a language and* $\mathcal{D} = \{\mathcal{D}_n\}_n$ *is an efficiently sampleable collection of distributions over* $\{0,1\}^n$*.*

Definition 9 (Infinitely-Often One-Sided Average-Case BPP)**.** *Let* $\mathsf{io1AvgBPP}$ *be the class of average-case problems* $(\mathcal{L}, \mathcal{D})$ *such that there exists a PPT algorithm* $\mathcal{A}$*, functions* $a, b : \mathbb{N} \to [0,1]$*, and a polynomial* p *such that for infinitely many* n*,*

- *For all* $x \in \mathcal{L} \cap \{0,1\}^n$, $\Pr[\mathcal{A}(x) = 1] \geq a(n)$
- $\Pr\left[\mathcal{A}(x) = 1 \middle| \begin{array}{c} x \xleftarrow{\$} \mathcal{D}_n \\ x \notin \mathcal{L} \end{array}\right] \leq b(n)$
- $a(n) - b(n) \geq \frac{1}{p(n)}$

We let $\mathsf{io1AvgBPP/poly}$ *be defined similarly, except that* $\mathcal{A}$ *is allowed to be non-uniform.*

Remark 6. *We note that the above definitions only require a decision algorithm to get some noticeable gap in acceptance probability between instances in the language and those not in the language. For* ioBPP *and* $\mathsf{ioBPP/poly}$*, standard amplification techniques can show that this is equivalent to a "constant gap" definition that sets* $a(n) = \frac{2}{3}$ *and* $b(n) = \frac{1}{3}$*. However, these techniques don't work with average-case guarantees, and hence we don't expect a similar statement to hold for* $\mathsf{io1AvgBPP}$ *nor* $\mathsf{io1AvgBPP/poly}$*.*

4 Universal Approximation with One-Sided Error

4.1 Definitions

Definition 10 (Infinitely-Often One-Sided Error Universal Approximation)**.** *Fix an alphabet* Σ*. We say that Infinitely-Often One-Sided Error Universal Approximation (io-1UA) is possible if for every sampler* M *from* m_i *input bits to* m_o *output bits, all polynomials* (p_1, p_2)*, and all noticeable functions* α*, there exists a non-uniform PPT machine* N *such that for infinitely many* n *and all* $x \in \Sigma^n$*,*

1. $\Pr\Big[N(x, M(x;r)) < (1 - \alpha(n))p_{M(x;r)} | r \xleftarrow{\$} \{0,1\}^{m_i(n)}\Big] \leq \frac{1}{p_1(n)}$
2. *For all* $y \in \{0,1\}^{m_o(n)}$, $\Pr[N(x,y) > (1 + \alpha(n))p_y] \leq \frac{1}{p_2(n)}$

where $p_y := \Pr\Big[M(x;r) = y | r \xleftarrow{\$} \{0,1\}^{m_i(n)}\Big]$*.*

4.2 No Weak OWF Implies io-1UA

In this section, we show that if weak one-way functions don't exist, io-1UA is possible. The key tool is a form of "coarse" estimation implicit in [Imp92]: as long as there are no weak one-way functions, we can (infinitely-often) estimate the number of random strings that lead to a given output, though we may overestimate by a polynomial factor. In order to get a closer approximation, we use the coarse estimate to choose a number of pairwise-independent hash bits to append to the function such that (with high probability) the number of hash outputs is a slightly larger than the number of preimages for the y we're trying to estimate. By choosing the number of bits carefully, we can ensure that the number of hash outputs is high enough that "most" preimages of y have unique hashes, but also small enough that we still have a noticeable probability of choosing such a hash output if we sample one at random. The latter guarantee (along with the non-existence of weak one-way functions) allows us to estimate the number of hashes that are mapped to by preimages of y by random sampling; the former guarantee says that this is close to the number of preimages of y.

Theorem 8. *Fix an alphabet Σ. If weak one-way functions don't exist, io-1UA is possible.*

Proof. Fix a sampler M, polynomials (p_1, p_2), and a noticeable function α.[7] In order to construct an estimator N_F for these parameters, we will need a few building blocks:

- A hash family $\{H_n\}_{n\in\mathbb{N}}$ where to sample $h \xleftarrow{\$} H_n$, one samples $2m_i(n)$ pairwise-independent hashes from $\{0,1\}^{m_i(n)}$ to $\{0,1\}$ and concatenates their outputs. The hash keys themselves are uniformly random strings in $\{0,1\}^{2m_i^2(n)}$.
- A deterministic machine $\tilde{M}$ such that $\tilde{M}(x,h,i,r) = (h,i,M(x;r),h(r)[:i])$, where $x \in \Sigma^*$ is an auxiliary input, h is a description of a hash from $H_{|x|}$, i is an index in $[2m_i(|x|)]$, and r is a string in $\{0,1\}^{m_i(|x|)}$.
- A non-uniform PPT machine $\tilde{M}^{-1}$ such that for infinitely many n, we have that for all $x \in \Sigma^n$,

$$\Pr\left[\begin{array}{l} M(x;\tilde{r}) = y \\ h(\tilde{r})[:i] = z \end{array} \middle| \begin{array}{c} r \xleftarrow{\$} \{0,1\}^{m_i(n)} \\ y \leftarrow M(x;r) \\ h \xleftarrow{\$} H_n \\ i \xleftarrow{\$} [2m_i(n)] \\ z \leftarrow h(r)[:i] \\ \tilde{r} \leftarrow \tilde{M}^{-1}(x,h,i,y,z) \end{array}\right] \geq 1 - \frac{\alpha(n)}{144 \cdot m_i(n) \cdot p_1(n)} \tag{5}$$

  Such a machine is guaranteed by the non-existence of weak one-way functions.[8]

[7] WLOG, we will assume that $\alpha(n) < 1$ for all n. If this is not the case, we can replace $\alpha(n)$ with $\alpha'(n) := \min(1, \alpha(n))$. Since $\alpha'(n) \leq \alpha(n)$, building io-1UA with respect to α' also satisfies the definition with respect to α.

[8] Technically, the inverter thus guaranteed would also output $\tilde{h}$ and $\tilde{i}$, but the inverter can only be better if we force it to always output $\tilde{h} = h$ and $\tilde{i} = i$. Thus, for ease of syntax, we ignore $\tilde{h}$ and $\tilde{i}$.

- A constant c such that for sufficiently large n, $n^c > \frac{72p_1(n)}{\alpha(n)}$. Such a constant must exist because p_1 is a polynomial and α is noticeable.

We will also use the following result, adapted for our purposes from [Imp92].

Lemma 1 (Implicit in [Imp92], Theorem 4.2.2)**.** *There exists a non-uniform PPT machine N_C such that for any sufficiently large n where (5) holds and all $x \in \Sigma^n$,*

1. $\Pr\left[N_C(x, M(x;r)) < k_{M(x;r)} | r \xleftarrow{\$} \{0,1\}^{m_i(n)}\right] \leq \frac{1}{9p_1(n)}$
2. *For all $y \in \{0,1\}^{m_o(n)}$ and all polynomials q,* $\Pr[N_C(x,y) > q(n) \cdot k_y] \leq \frac{2n \cdot m_i(n)}{q(n)}$

where $k_y := |\{r \in \{0,1\}^{m_i(n)} | M(x;r) = y\}|$.

For completeness, we include a proof of this lemma Appendix B. However, we note that it is essentially just a rephrasing of the proof from [Imp92]. With these building blocks in mind, we give our construction of N_F in Algorithm 1.

Algorithm 1: Fine estimator N_F

Input: $x \in \Sigma^n$, $y \in \{0,1\}^{m_o(n)}$

Output: $\tilde{p}_y$, estimate of $\Pr\left[M(x;r) = y | r \xleftarrow{\$} \{0,1\}^{m_i(n)}\right]$

Compute $\tilde{k} \leftarrow N_C(x,y)$

Let $t = \frac{288}{\alpha(n)^2} \cdot n^{c+1} \cdot m_i(n) \cdot p_1(n)^2 \cdot p_2(n)^2$, $i^* = \log \tilde{k} + c \log n$, and numSuccess $= 0$

Sample $h \xleftarrow{\$} H_n$

for $\_ \leftarrow 1$ **to** t **do**

  Sample $z \xleftarrow{\$} \{0,1\}^{i^*}$

  Compute $\tilde{r} \leftarrow \tilde{M}^{-1}(x, h, i^*, y, z)$

  if $M(x;\tilde{r}) = y$ ***and*** $h(\tilde{r})[:i^*] = z$ **then**

    numSuccess $=$ numSuccess $+ 1$

  end

end

Output $\tilde{p}_y = \frac{\text{numSuccess}}{t} \cdot \frac{n^c \cdot \tilde{k}}{2^{m_i(n)}}$

In order to prove that N_F satisfies the first guarantee of Definition 10, we define three "bad" events that can cause N_F to output a bad estimate and show that they all happen with small probability. Specifically, for any $x \in \Sigma^n$, we define the following events over the randomness of sampling $y = M(x;r)$ and the internal randomness of N_F:

- BadInv as the event that $p_{y,\tilde{k},h} < (1 - \frac{\alpha(n)}{2}) \cdot \frac{k_y}{n^c \cdot \tilde{k}}$,
- BadCoarse as the event that $\tilde{k} > 6n \cdot m_i(n) \cdot p_1(n) \cdot k_y$, and
- BadSamp as the event that $\frac{\text{numSuccess}}{t} < (1 - \frac{\alpha(n)}{2}) p_{y,\tilde{k},h}$

where $k_y := |\{r \in \{0,1\}^{m_i(n)} | M(x;r) = y\}|$ and

$$p_{y,\tilde{k},h} := \Pr\left[\begin{matrix} M(x;\tilde{r}) = y \\ h(\tilde{r})[:i^*] = z \end{matrix} \middle| \begin{matrix} z \xleftarrow{\$} \{0,1\}^{i^*} \\ \tilde{r} \leftarrow \tilde{M}^{-1}(x,h,i^*,y,z) \end{matrix}\right]$$

Note that if none of these bad events occur, we have that

$$\frac{\text{numSuccess}}{t} \geq \left(1 - \frac{\alpha(n)}{2}\right)^2 \cdot \frac{k_y}{n^c \cdot \tilde{k}} \geq (1 - \alpha(n)) \cdot \frac{k_y}{n^c \cdot \tilde{k}}$$

Thus, our final output $\tilde{p}_y$ will be at least $(1-\alpha(n)) \cdot \frac{k_y}{2^{m_i(n)}}$, which is precisely $(1-\alpha(n)) \cdot p_y$. Combining this with the following three claims, we have that for sufficiently large n such that (5) holds and all $x \in \Sigma^n$, N_F satisfies the first guarantee of Definition 10.

Claim 1. *For sufficiently large n such that (5) holds and all $x \in \Sigma^n$, $\Pr[BadInv] \leq \frac{1}{3p_1(n)}$.*

Claim 2. *For sufficiently large n such that (5) holds and all $x \in \Sigma^n$, $\Pr[BadCoarse] \leq \frac{1}{3p_1(n)}$.*

Claim 3. *For all n and all $x \in \Sigma^n$, $\Pr\left[BadSamp | \overline{BadInv} \wedge \overline{BadCoarse}\right] \leq \frac{1}{3p_1(n)}$.*

Proof of Claim 1. Fix an n such that (5) holds and an $x \in \Sigma^n$. There are three possible events that could cause BadInv to happen:

1. $\tilde{k} < k_y$
2. The h we choose causes too many preimages of y to collide. More formally, say that r such that $M(x;r) = y$ collides under h if there exists an $r' \in \{0,1\}^{m_i(n)}$ with $r' \neq r$ such that $M(x;r') = y$ and $h(r')[:i^*] = h(r)[:i^*]$. Then our bad event is if

$$\Pr\left[r \text{ collides under } h | r \xleftarrow{\$} R_y\right] > \frac{\alpha(n)}{8}$$

   where $R_y := \{r \in \{0,1\}^{m_i(n)} | M(x;r) = y\}$.
3. $\tilde{M}^{-1}$ does poorly on the sampled y, $\tilde{k}$, and h. More formally, say that $\tilde{M}^{-1}$ fails on input (x,h,i,y,z) if its output $\tilde{r}$ has either $M(x;\tilde{r}) \neq y$ or $h(\tilde{r})[:i] \neq z$. Then our bad event is if

$$\Pr\left[\tilde{M}^{-1} \text{ fails on } (x,h,i^*,y,h(r)[:i^*]) | r \xleftarrow{\$} R_y\right] > \frac{\alpha(n)}{8}$$

   where $R_y := \{r \in \{0,1\}^{m_i(n)} | M(x;r) = y\}$.

First, note that by Lemma 1, the first bad event happens with probability at most $\frac{1}{9p_1(n)}$ as long as n is sufficiently large.

Next, we turn to the second bad event and bound the probability of it happening conditioned on the first bad event not happening (ie, on $\tilde{k} \geq k_y$). By the pairwise independence of H_n, the probability that any $r, r' \in \{0,1\}^{m_i(n)}$ have the same first i^* hash bits is $2^{-i^*} = \frac{1}{\tilde{k}} \cdot \frac{1}{n^c}$. Applying a union bound and the fact that we are conditioning on $\tilde{k} \geq k_y$, we get that over a random $h \xleftarrow{\$} H_n$, the probability that any $r \in R_y$ colliding under h is at most $\frac{1}{n^c}$, which

by our definition of c is at most $\frac{\alpha(n)}{72p_1(n)}$ as long as n is sufficiently large. In particular, this will also hold if we sample a random r from R_y. A Markov bound now tells us that as long as n is sufficiently large,

$$\Pr\left[\Pr\left[r \text{ collides under } h | r \xleftarrow{\$} R_y\right] > \frac{\alpha(n)}{8} \middle| h \xleftarrow{\$} H_n\right] \leq \frac{1}{9p_1(n)}$$

Thus, the probability of the second bad event occurring conditioned on the first bad event not occurring is at most $\frac{1}{9p_1(n)}$ as long as n is large enough.

Finally, we consider the third bad event. Note that in (5), we can switch the order of sampling r and y without changing the distribution; that is, we can first get a random $y \leftarrow M(x)$ and then sample r from R_y. Applying a Markov bound to this, we get that

$$\Pr\left[\Pr\left[\tilde{M}^{-1} \text{ fails on } (x,h,i,y,z) \middle| \begin{array}{c} i \xleftarrow{\$} [2m_i(n)] \\ r \xleftarrow{\$} R_y \\ z = h(r)[:i] \end{array}\right] > \frac{\alpha(n)}{16m_i(n)} \middle| \begin{array}{c} y \leftarrow M(x) \\ h \xleftarrow{\$} H_n \end{array}\right] \leq \frac{1}{9p_1(n)}$$

If we sample y and h such that this does not happen (ie, the probability that $\tilde{M}^{-1}$ fails over i and z is at most $\frac{\alpha(n)}{16m_i(n)}$), for any fixed i the probability over z that $\tilde{M}^{-1}$ fails must be at most $\frac{\alpha(n)}{8}$ as there are only $2m_i(n)$ possible values for i. In particular, this holds for $i = i^*$, so such a y and h avoid the third bad event. Putting this all together, the probability of the third bad event happening is at most $\frac{1}{9p_1(n)}$.

Combining the above, we have that the probability that at least one of the bad events occurs is at most $\frac{1}{3p_1(n)}$ for sufficiently large n such that (5) holds. Thus, to prove the claim, it suffices to show that if none of the bad events occur, BadInv cannot occur either.

For the sake of analysis, consider an (inefficient, non-uniform) machine $U(x,h,i,y,z)$ that is identical to $\tilde{M}^{-1}$, except that if there exist distinct $r, r' \in R_y$ such that $h(r)[:i] = h(r')[:i] = z$, U outputs $\perp$. Since we are in a setting where the second bad event doesn't happen, we know that the success probability of U over $z = h(r)[:i^*]$ for a random $r \xleftarrow{\$} R_y$ can be at worst $\frac{\alpha(n)}{8}$ lower than the success probability for $\tilde{M}^{-1}$. Also applying that the third bad event doesn't happen, we get that

$$\Pr\left[U \text{ fails on } (x,h,i^*,y,z) \middle| \begin{array}{c} r \xleftarrow{\$} R_y \\ z = h(r)[:i^*] \end{array}\right] \leq \frac{\alpha(n)}{4} \tag{6}$$

Let Z_U denote the set of all non-colliding hash values; that is,

$$Z_U := \{z | \exists \text{ unique } r \in \{0,1\}^{m_i(n)} \text{ st } M(x;r) = y \wedge h(r)[:i^*] = z\}$$

Note that if we condition on $z \in Z_U$ in (6), the probability of U failing can only decrease, since U is by construction guaranteed to fail on any $z \notin Z_U$. Furthermore, since each $z \in Z_U$ has exactly one $r \in \{0,1\}^{m_i(n)}$ that maps to it, sampling z conditioned on $z \in Z_U$ is equivalent to simply sampling a random element from Z_U. Thus, we have that

$$\Pr\left[U \text{ fails on } (x,h,i^*,y,z) \middle| z \xleftarrow{\$} Z_U\right] \leq \frac{\alpha(n)}{4}$$

Now if we instead sample $z \xleftarrow{\$} \{0,1\}^{i^*}$, that can be viewed as sampling from Z_U with probability $\frac{|Z_U|}{\tilde{k} \cdot n^c}$ and from $\overline{Z_U}$ otherwise. Noting that $|Z_U| \geq (1 - \frac{\alpha(n)}{8}) \cdot k_y$ because we're conditioning on the second bad event not happening, we get that

$$\begin{aligned}
\Pr\left[U \text{ fails on } (x,h,i^*,y,z) \middle| z \xleftarrow{\$} \{0,1\}^{i^*}\right] &\leq \left(1 - \frac{|Z_U|}{\tilde{k} \cdot n^c}\right) + \Pr\left[U \text{ fails on } (x,h,i^*,y,z) \middle| z \xleftarrow{\$} Z_U\right] \cdot \frac{|Z_U|}{\tilde{k} \cdot n^c} \\
&\leq 1 - \left(1 - \frac{\alpha(n)}{4}\right) \frac{|Z_U|}{\tilde{k} \cdot n^c} \\
&\leq 1 - \left(1 - \frac{\alpha(n)}{4}\right)\left(1 - \frac{\alpha(n)}{8}\right) \frac{k_y}{\tilde{k} \cdot n^c} \\
&\leq 1 - \left(1 - \frac{\alpha(n)}{2}\right) \frac{k_y}{\tilde{k} \cdot n^c}
\end{aligned}$$

Finally, we note that

$$\begin{aligned}
p_{y,\tilde{k},h} &= 1 - \Pr\left[\tilde{M}^{-1} \text{ fails on } (x,h,i^*,y,z) \middle| z \xleftarrow{\$} \{0,1\}^{i^*}\right] \\
&\geq 1 - \Pr\left[U \text{ fails on } (x,h,i^*,y,z) \middle| z \xleftarrow{\$} \{0,1\}^{i^*}\right] \\
&\geq \left(1 - \frac{\alpha(n)}{2}\right) \frac{k_y}{\tilde{k} \cdot n^c}
\end{aligned}$$

where in the second line we used the fact that U can only fail more often than $\tilde{M}^{-1}$. Thus, we indeed have that we avoid BadInv as long as none of the three bad events happen. □

Proof of Claim 2. This follows from the second guarantee on N_C from Lemma 1, applied with respect to $q(n) = 6n \cdot m_i(n) \cdot p_1(n)$. □

Proof of Claim 3. The event BadSamp occurs when our observed probability of $\tilde{M}^{-1}$ successfully inverting is much smaller than the actual probability of it doing so. Noting that the observed number of successes is simply a binomial random variable with paramters $(t, p_{y,\tilde{k},h})$, a Chernoff bound immediately gives us that $\Pr[\text{BadSamp}] \leq \exp\left(-\frac{\alpha(n)^2 \cdot t \cdot p_{y,\tilde{k},h}}{8}\right)$. Conditioning on BadInv and BadCoarse not happening, we know that

$$p_{y,\tilde{k},h} \geq \left(1 - \frac{\alpha(n)}{2}\right) \cdot \frac{k_y}{n^c \cdot \tilde{k}} \geq \left(1 - \frac{\alpha(n)}{2}\right) \frac{1}{6n^{c+1} \cdot m_i(n) \cdot p_1(n)} \geq \frac{1}{12n^{c+1} \cdot m_i(n) \cdot p_1(n)}$$

where in the last inequality we used that $\alpha(n) \leq 1$. Plugging this in as well as our value of t, we get that

$$\Pr\left[\text{BadSamp} | \overline{\text{BadInv}} \wedge \overline{\text{BadCoarse}}\right] \leq \exp\left(-3p_1(n) \cdot p_2(n)^2\right) \leq \frac{1}{3p_1(n)}$$

□

It now only remains to prove that N_F satisfies the second guarantee of Definition 10. Fix any n such that (5) holds, any $x \in \Sigma^n$ as auxiliary input, and any $y \in \{0,1\}^{m_o(n)}$ as input

to N_F. Note that because $\tilde{M}^{-1}$ can only successfully invert if we choose a z that is actually the hash of some r such that $M(x;r) = y$, we know that $p_{y,\tilde{k},h} \leq \frac{k_y}{n^c \cdot \tilde{k}}$ regardless of the values of y, $\tilde{k}$, and h. Since our probability of overestimating only goes up as $p_{y,\tilde{k},h}$ increases, for the purposes of our bound we can pessimistically assume that $p_{y,\tilde{k},h} = \frac{k_y}{n^c \cdot \tilde{k}}$.[9]

With this in place, bounding the probability that $\tilde{p}_y > (1+\alpha(n))p_y$ is equivalent to bounding the probability that numSuccess is at least $(1+\alpha(n))$ times its expectation. Applying a Chernoff bound, this is at most $\exp\left(-\frac{\alpha(n)^2 \cdot t \cdot k_y}{3n^c \cdot \tilde{k}}\right)$. In order to ensure that this bound is small, we need to make sure that $\tilde{k}$ is not too large; by the second guarantee of Lemma 1 with respect to $q(n) = 4n \cdot m_i(n) \cdot p_2(n)$, we have that as long as n is large enough, $\Pr\left[\tilde{k} > 4n \cdot m_i(n) \cdot p_2(n) \cdot k_y\right] \leq \frac{1}{2p_2(n)}$. Conditioning on this bad event not happening, our bound becomes

$$\begin{aligned}
\Pr\left[\tilde{p}_y \geq (1+\alpha(n))p_y | \tilde{k} \leq 4n \cdot m_i(n) \cdot p_2(n) \cdot k_y\right] &\leq \exp\left(-\frac{\alpha(n)^2 \cdot t}{12n^{c+1} \cdot m_i(n) \cdot p_2(n)}\right) \\
&= \exp\left(-24p_1(n)^2 \cdot p_2(n)\right) \\
&\leq \frac{1}{2p_2(n)}
\end{aligned}$$

where the second line comes from plugging in our chosen value of t. Putting this all together, we have that as long as n is sufficiently large,

$$\begin{aligned}
\Pr[\tilde{p}_y \geq (1+\alpha(n))p_y] &\leq \Pr\left[\tilde{p}_y \geq (1+\alpha(n))p_y | \tilde{k} \leq 4n \cdot m_i(n) \cdot p_2(n) \cdot k_y\right] \\
&\quad + \Pr\left[\tilde{k} > 4n \cdot m_i(n) \cdot p_2(n) \cdot k_y\right] \\
&\leq \frac{1}{2p_2(n)} + \frac{1}{2p_2(n)} = \frac{1}{p_2(n)}
\end{aligned}$$

Putting this all together, we have that N_F satisfies both conditions of Definition 10 on sufficiently large n such that (5) holds and all $x \in \Sigma^n$. Since (5) holds for infinitely many n, we have that N_F satisfies both conditions infinitely often, as desired.

□

Remark 7. *Similar to [Imp92], our proof only actually use the non-existence of weak one-way functions to get a sufficiently good inverter for $\tilde{M}$. Thus, one can rephrase our result as saying that for every sampler M, either we can do io-1UA for M or $\tilde{M}$ is a weak one-way function.*

Remark 8. *Note that while the algorithm N_F we construct is non-uniform, this is only because it (and the coarse estimator N_C from [Imp92]) needs to use the ability to break weak one-way function candidates—and the non-existence of weak one-way functions only guarantees us that this is possible with a non-uniform adversary. Thus our proof is, in a sense,* uniformity-preserving*: assuming all weak one-way function candidates can be broken by a uniform adversary, io-1UA is possible with a uniform PPT machine.*

[9] Technically, this is only a valid probability if $k_y \leq n^c \cdot \tilde{k}$. But note that the maximum value N_F can output is $\frac{n^c \cdot \tilde{k}}{2^{m_i(n)}}$. Thus, if $k_y > \tilde{k} \cdot n^c$, N_F is guaranteed to output at most $\frac{k_y}{2^{m_i(n)}}$, and so trivially satisfies the second guarantee.

Remark 9. *The non-existence of one-way functions implies that* both *1-universal approximation and universal extrapolation are successful* together, infinitely often, even when applied to different efficient samplers *as long as the number of samplers is independent of the security parameter. That is, there are infinitely many security parameters on which all of these approximation/extrapolation tasks succeed together on any finite set of samplers. This is because one-way function combiners imply the existence of a* single *one-way function, such that there is a security-parameter preserving reduction from the task of inverting the combined one-way function to performing each of the underlying approximation/extrapolation tasks.*

5 OWF From Worst-Case Hardness

In this section, we show that the results of [HN24] can be extended to work with NIZKs that have non-negligible errors; that is, we show that as long as NP is hard *in the worst case* and has NIZKs with errors in certain regimes, one-way functions exist. To prove this, we follow the structure of the simplified proof given in [LMP24], which proceeds in three steps:

1. Showing that a (high-error) NIZK for a worst-case hard language gives auxiliary input one-way functions.
2. Showing that auxiliary-input one-way functions imply the existence of a language in NP that is hard for what we call "one-sided average-case" algorithms, which we define below.[10]
3. Showing that a (high-error) NIZK for a one-sided average-case hard language gives a one-way function.

For the first and third steps, we in fact show two variants. The former adapts techniques from [OW93] to show that it suffices to work with $(\epsilon_{\mathsf{c}}, \epsilon_{\mathsf{s}}, \epsilon_{\mathsf{zk}})$-NIZKs for which $\epsilon_{\mathsf{c}} + \epsilon_{\mathsf{s}} + 2\epsilon_{\mathsf{zk}}$ is noticeably less than 1; the latter uses our new notion of io-1UA to show that $\epsilon_{\mathsf{c}} + \epsilon_{\mathsf{zk}} + 2\sqrt{\epsilon_{\mathsf{s}}}$ being noticeably less than 1 suffices (note that these two bounds are incomparable). As a final optimization, we show in Section 5.5 that one can adapt amplification results from [BG24] to remove the factor 2 in the latter bound as long as $\epsilon_{\mathsf{c}} = o(1)$.

5.1 Part One: ai-OWF from Worst-Case Hardness

5.1.1 OW Construction

We begin with the simpler proof, based on techniques from [OW93].

Lemma 2. *Suppose there exists a language $\mathcal{L} \notin \mathsf{ioP/poly}$ such that $\mathcal{L}$ has an $(\epsilon_{\mathsf{c}}, \epsilon_{\mathsf{s}}, \epsilon_{\mathsf{zk}})$-NIZK with $\epsilon_{\mathsf{c}} + \epsilon_{\mathsf{s}} + 2\epsilon_{\mathsf{zk}} <_n 1$. Then there exists an auxiliary-input one-way function.*

[10] In fact, [LMP24] used a more standard notion of "zero-sided average-case" algorithms, but in order for our proof in the next step to go through we need to allow average-case guarantees when $x \notin \mathcal{L}$.

Proof. Let $(\mathrm{Gen}, \mathrm{P}, \mathrm{V})$ be the $(\epsilon_{\mathsf{c}}, \epsilon_{\mathsf{s}}, \epsilon_{\mathsf{zk}})$-NIZK for $\mathcal{L}$ guaranteed by the theorem statement, and $p(n)$ be a polynomial such that for sufficiently large n, $\epsilon_{\mathsf{c}}(n) + \epsilon_{\mathsf{s}}(n) + 2\epsilon_{\mathsf{zk}}(n) < 1 - \frac{1}{p(n)}$. Suppose for the sake of contradiction that auxiliary-input one-way functions don't exist. Define:

- M as an auxiliary-input sampler where $M(x; r)$ computes $(\mathsf{crs}, \pi) \leftarrow \mathrm{Sim}(x; r)$ and outputs crs.
- N as the io-UE machine guaranteed by Theorem 6 with respect to sampler M and polynomial $2p(n)$.

We now claim that Algorithm 2 decides $\mathcal{L}$ infinitely often, thus contradicting that $\mathcal{L} \notin \mathsf{ioP/poly}$.

Algorithm 2: Algorithm $\mathcal{A}$ to decide $\mathcal{L}$

Input: $x \in \{0,1\}^n$
Output: Decision bit b, with $b = 1$ representing $x \in \mathcal{L}$

Compute $\mathsf{crs} \leftarrow \mathrm{Gen}(1^n)$
Compute $r' \leftarrow N(x, \mathsf{crs})$
Compute $(\mathsf{crs}', \pi') \leftarrow \mathrm{Sim}(x; r')$
if $\mathsf{crs}' \neq \mathsf{crs}$ **then**
  Output $b = 0$
else
  Output $b = V(\mathsf{crs}', x, \pi')$
end

First, note that for any n, $\mathcal{A}$ accepts every $x \in \{0,1\}^n - \mathcal{L}$ with probability at most $\epsilon_{\mathsf{s}}(n)$. Indeed, $\mathcal{A}$ can only accept x if it finds an accepting proof for x with respect to a CRS generated by $\mathrm{Gen}(1^n)$. Since $\mathcal{A}$ is an efficient (non-uniform) machine, the soundness of our NIZK says that this can occur with probability at most $\epsilon_{\mathsf{s}}(n)$.

Next, we consider what $\mathcal{A}$ does when $x \in \mathcal{L}$. For this case, we define two hybrid algorithms, below as Algorithms 3 and 4.

Algorithm 3: Hybrid algorithm $\mathcal{A}_1$

Input: $x \in \{0,1\}^n$
Output: Decision bit b, with $b = 1$ representing $x \in \mathcal{L}$

Compute $(\mathsf{crs}, \pi) \leftarrow \mathrm{Sim}(x; r)$ for a random r
Compute $r' \leftarrow N(x, \mathsf{crs})$
Compute $(\mathsf{crs}', \pi') \leftarrow \mathrm{Sim}(x; r')$
if $\mathsf{crs}' \neq \mathsf{crs}$ **then**
  Output $b = 0$
else
  Output $b = V(\mathsf{crs}', x, \pi')$
end

Algorithm 4: Hybrid algorithm $\mathcal{A}_2$

Input: $x \in \{0,1\}^n$

Output: Decision bit b, with $b = 1$ representing $x \in \mathcal{L}$

1 Compute $(\mathsf{crs}, \pi) \leftarrow \mathrm{Sim}(x; r)$ for a random r

2 Output $b = V(\mathsf{crs}, x, \pi)$

Fix any n such that the guarantees of Definition 4 on N hold and any $x \in \{0,1\}^n \cap \mathcal{L}$. Let p_i be the probability that $\mathcal{A}_i(x) = 1$, where $\mathcal{A}_0 := \mathcal{A}$. We first claim that $|p_0 - p_1| \leq \epsilon_{\mathsf{zk}}(n)$. This follows immediately from zero-knowledge, since the only difference between $\mathcal{A}_0$ and $\mathcal{A}_1$ is that $\mathcal{A}_0$ starts with an honestly-generated CRS, whereas $\mathcal{A}_1$ starts with a simulated CRS.

Next, we claim that $|p_1 - p_2| \leq \frac{1}{2p(n)}$. Indeed, from Definition 4, we know that the distributions of $r \| \mathsf{crs}$ and $r' \| \mathsf{crs}$ are $\frac{1}{2p(n)}$-close. But note that if we replace r' with r, $\mathcal{A}_1$ becomes equivalent to $\mathcal{A}_2$, and hence our claim follows.

Finally consider $\mathcal{A}_2$. Note that if we replace the first line with and honestly-generated CRS and proof, completeness tells us that the second line would accept with probability at least $1 - \epsilon_{\mathsf{c}}(n)$. Thus, by zero-knowledge we have that $p_2 \geq 1 - \epsilon_{\mathsf{c}}(n) - \epsilon_{\mathsf{zk}}(n)$. Combining this with the above, we have that $p_0 \geq 1 - \epsilon_{\mathsf{c}}(n) - 2\epsilon_{\mathsf{zk}}(n) - \frac{1}{2p(n)}$.

Putting this all together, we have that in Definition 6, we can set $a(n) = 1 - \epsilon_{\mathsf{c}}(n) - 2\epsilon_{\mathsf{zk}}(n) - \frac{1}{2p(n)}$ and $b(n) = \epsilon_{\mathsf{s}}(n)$ to satisfy the first two conditions infinitely often. We then note that for sufficiently large n,

$$\begin{aligned} a(n) - b(n) &= 1 - (\epsilon_{\mathsf{c}}(n) + \epsilon_{\mathsf{s}}(n) + 2\epsilon_{\mathsf{zk}}(n)) - \frac{1}{2p(n)} \\ &\geq 1 - \left(1 - \frac{1}{p(n)}\right) - \frac{1}{2p(n)} = \frac{1}{2p(n)} \end{aligned}$$

which satisfies the third condition. Thus, we have that $\mathcal{L} \in \mathsf{ioBPP/poly}$ (and so by Remark 5, $\mathsf{ioP/poly}$), which gives our desired contradiction.

□

5.1.2 Our Construction

We now give a slightly more technical proof using our new notion of io-1UA to get parameters that can work even if ϵ_{zk} is larger than $\frac{1}{2}$.

Lemma 3. *Suppose there exists a language $\mathcal{L} \notin \mathsf{ioP/poly}$ such that $\mathcal{L}$ has an $(\epsilon_{\mathsf{c}}, \epsilon_{\mathsf{s}}, \epsilon_{\mathsf{zk}})$-NIZK with $\epsilon_{\mathsf{c}} + \epsilon_{\mathsf{zk}} + 2\sqrt{\epsilon_{\mathsf{s}}} <_n 1$. Then there exists an auxiliary-input one-way function.*

Proof. Suppose for the sake of contradiction that auxiliary-input one-way functions don't exist. From this we will build a non-uniform PPT algorithm to decide $\mathcal{L}$ infinitely often, which will contradict that $\mathcal{L} \notin \mathsf{ioP/poly}$. Our algorithm will use the following building blocks:

- $(\mathrm{Gen}, \mathrm{P}, \mathrm{V})$ as the $(\epsilon_{\mathsf{c}}, \epsilon_{\mathsf{s}}, \epsilon_{\mathsf{zk}})$-NIZK for $\mathcal{L}$, with Sim as the simulator guaranteed by zero-knowledge.

- p as a polynomial such that $\epsilon_{\mathsf{c}}(n) + \epsilon_{\mathsf{zk}}(n) + 2\sqrt{\epsilon_{\mathsf{s}}(n)} < 1 - \frac{1}{p(n)}$ for sufficiently large n. We will assume WLOG that $p(n) \geq 1$ for all $n \in \mathbb{N}$.
- M_{real} as an auxiliary input sampler where $M_{\text{real}}(x; r) = \text{Gen}(1^{|x|}; r)$.
- N_{real} as the io-1UA estimator guaranteed by Theorem 8 with respect to $\Sigma = \{0,1\}$, sampler M_{real}, polynomials $q_1(n) = q_2(n) = 20p(n)$, and $\alpha = \frac{1}{20p(n)}$.
- M_{sim} as an auxiliary-input sampler where $M(x; r)$ computes $(\mathsf{crs}, \pi) \leftarrow \text{Sim}(x; r)$ and outputs crs.
- N_{sim} as the io-1UA estimator guaranteed by Theorem 8 with respect to $\Sigma = \{0,1\}$, sampler M_{sim}, polynomials $q_1(n) = q_2(n) = 20p(n)$, and $\alpha = \frac{1}{20p(n)}$.
- R_{sim} as the io-UE machine guaranteed by Theorem 6 with respect to sampler M_{sim} and polynomial $20p(n)$.[11]
- Γ as the set of all n such that the guarantees on N_{real}, N_{sim}, and R_{sim} all hold. By Remark 9, Γ is infinite.

With these building blocks, we give our algorithm $\mathcal{A}$ for $\mathcal{L}$ in Algorithm 5.

Algorithm 5: Algorithm $\mathcal{A}$ to decide $\mathcal{L}$

Input: $x \in \{0,1\}^n$
Output: Decision bit b, with $b = 1$ representing $x \in \mathcal{L}$

Compute $(\mathsf{crs}, \pi) \leftarrow \text{Sim}(x)$
Compute $\tilde{p}_{\text{sim}} \leftarrow N_{\text{sim}}(x, \mathsf{crs})$
Compute $\tilde{p}_{\text{real}} \leftarrow N_{\text{real}}(x, \mathsf{crs})$
if $\tilde{p}_{sim} > \frac{20p(n)+1}{20p(n)-1} \cdot \frac{1}{\sqrt{\epsilon_{\mathsf{s}}(n)}} \cdot \tilde{p}_{real}$ **then**
  Output $b = 0$
else
  Output $b = V(\mathsf{crs}, x, \pi)$
end

The core of our proof is in the following two claims:

Claim 4. *For all $n \in \Gamma$ and $x \in \mathcal{L} \cap \{0,1\}^n$,*

$$\Pr[\mathcal{A}(x) = 1] \geq 1 - \epsilon_{\mathsf{zk}}(n) - \epsilon_{\mathsf{c}}(n) - \sqrt{\epsilon_{\mathsf{s}}(n)} - \frac{1}{10p(n)}$$

Claim 5. *For all $n \in \Gamma$ and $x \in \{0,1\}^n - \mathcal{L}$,*

$$\Pr[\mathcal{A}(x) = 1] \leq \sqrt{\epsilon_{\mathsf{s}}(n)} + \frac{2}{5p(n)}$$

[11] Note that we do not use R_{sim} in the algorithm itself, only in the analysis thereof.

Now note that these two claims together tell us that $\mathcal{L} \in \mathsf{ioBPP/poly}$ (and so by Remark 5, $\mathcal{L} \in \mathsf{ioP/poly}$). Indeed, if we set $a(n) = 1 - \epsilon_{\mathsf{zk}}(n) - \epsilon_{\mathsf{c}}(n) - \sqrt{\epsilon_{\mathsf{s}}(n)} - \frac{1}{10p(n)}$ and $b(n) = \sqrt{\epsilon_{\mathsf{s}}(n)} + \frac{2}{5p(n)}$ in Definition 6, these two claims will tell us that we satisfy the first two conditions of the definition for any $n \in \Gamma$. For sufficiently large n, we have that

$$\begin{aligned} a(n) - b(n) &= 1 - \left(\epsilon_{\mathsf{c}}(n) + \epsilon_{\mathsf{zk}}(n) + 2\sqrt{\epsilon_{\mathsf{s}}(n)}\right) - \frac{1}{2p(n)} \\ &\geq 1 - \left(1 - \frac{1}{p(n)}\right) - \frac{1}{2p(n)} = \frac{1}{2p(n)} \end{aligned}$$

Since Γ is infinite, this tells us that there are infinitely many n such that all the conditions of Definition 6 are satisfied. Thus, all that remains is to prove Claims 4 and 5.

Proof of Claim 4. First, note that by zero-knowledge, the probability of $\mathcal{A}$ accepting x can change by at most $\epsilon_{\mathsf{zk}}(n)$ if we replace the (crs, π) in the first line with $\mathsf{crs} \leftarrow \mathrm{Gen}(1^n)$ and $\pi \leftarrow \mathrm{P}(\mathsf{crs}, x, w)$ for some witness w for x.

With this change in mind, let p_{sim} and p_{real} be the probabilities that $\tilde{p}_{\mathrm{sim}}$ and $\tilde{p}_{\mathrm{real}}$ are estimates of. That is, let p_{sim} be the probability of sampling crs from $\mathrm{Sim}(x)$ and p_{real} be the probability of sampling it from $\mathrm{Gen}(1^n)$. The core of our proof is in the following two subclaims.

Claim 6. *The probability that* $\tilde{p}_{sim} > \left(1 + \frac{1}{20p(n)}\right) p_{sim}$ *or* $\tilde{p}_{real} < \left(1 - \frac{1}{20p(n)}\right) p_{real}$ *is at most* $\frac{1}{10p(n)}$.

Proof. This follows directly from the guarantees on N_{sim} and N_{real} (the latter relying on the fact that crs is being sampled from $\mathrm{Gen}(1^n)$) and a union bound over the two bad events. □

Claim 7. *Suppose that* $\tilde{p}_{sim} \leq \left(1 + \frac{1}{20p(n)}\right) p_{sim}$ *and* $\tilde{p}_{real} \geq \left(1 - \frac{1}{20p(n)}\right) p_{real}$. *Then our modified algorithm can only reach line 5 if* $p_{sim} > \frac{1}{\sqrt{\epsilon_{\mathsf{s}}(n)}} \cdot p_{real}$.

Proof. In order to reach line 5, we must have that $\tilde{p}_{\mathrm{sim}} > \frac{20p(n)+1}{20p(n)-1} \cdot \frac{1}{\sqrt{\epsilon_{\mathsf{s}}(n)}} \cdot \tilde{p}_{\mathrm{real}}$. Plugging in our bounds on $\tilde{p}_{\mathrm{sim}}$ and $\tilde{p}_{\mathrm{real}}$, we get that this is only possible if

$$\left(1 + \frac{1}{20p(n)}\right) p_{\mathrm{sim}} > \frac{20p(n)+1}{20p(n)-1} \cdot \sqrt{\frac{1}{\epsilon_{\mathsf{s}}(n)}} \cdot \left(1 - \frac{1}{20p(n)}\right) p_{\mathrm{real}}$$

which after rearranging is equivalent to $p_{\mathrm{sim}} > \frac{1}{\sqrt{\epsilon_{\mathsf{s}}(n)}} \cdot p_{\mathrm{real}}$. □

But now note that because the sum over *all* possible crs of p_{sim} is 1, the probability of sampling a crs from $\mathrm{Gen}(1^n)$ such that $p_{\mathrm{sim}} > \frac{1}{\sqrt{\epsilon_{\mathsf{s}}(n)}} \cdot p_{\mathrm{real}}$ is at most $\sqrt{\epsilon_{\mathsf{s}}(n)}$.[12] Noting that our modified algorithm can only reject x if it reaches line 5 or has $\mathrm{V}(\mathsf{crs}, x, \pi) = 0$ (which happens with probability at most $\epsilon_{\mathsf{c}}(n)$ by completeness), a union bound gives us that our

[12] One can technically get a slightly tighter analysis by accounting for the fact that the real and simulated CRS distributions can be at most $\epsilon_{\mathsf{zk}}(n)$-far. However, it turns out that the improvement one gets from that is subsumed by the amplification we show in Section 5.5, so here we stick to the simpler proof.

modified algorithm can reject $x \in \mathcal{L}$ with probability at most $\epsilon_{\mathsf{c}}(n) + \sqrt{\epsilon_{\mathsf{s}}(n)} + \frac{1}{10p(n)}$, and so our main claim follows. □

Proof of Claim 5. For this proof, we consider three hybrids, each obtained by modifying how the algorithm $\mathcal{A}$ computes the CRS and proof it uses. The algorithm $\mathcal{A}_0$ used in hybrid 0 is identical to $\mathcal{A}$ as defined in Algorithm 5; the algorithms $\mathcal{A}_1$ and $\mathcal{A}_2$ used in hybrids 1 and 2 are defined in Algorithms 6 and 7, respectively.

Algorithm 6: Algorithm $\mathcal{A}_1$ for Hybrid 1

Input: $x \in \{0,1\}^n$
Output: Decision bit b, with $b = 1$ representing $x \in \mathcal{L}$

Compute $(\mathsf{crs}, \pi) \leftarrow \mathrm{Sim}(x; r)$ for a random r
Compute $r' \leftarrow R_{\mathrm{sim}}(x, \mathsf{crs})$
Compute $(\mathsf{crs}', \pi') \leftarrow \mathrm{Sim}(x; r')$
if $\mathsf{crs}' \neq \mathsf{crs}$ **then**
  Output $b = 0$
end
Compute $\tilde{p}_{\mathrm{sim}} \leftarrow N_{\mathrm{sim}}(x, \mathsf{crs}')$
Compute $\tilde{p}_{\mathrm{real}} \leftarrow N_{\mathrm{real}}(x, \mathsf{crs}')$
if $\tilde{p}_{sim} > \frac{20p(n)+1}{20p(n)-1} \cdot \frac{1}{\sqrt{\epsilon_{\mathsf{s}}(n)}} \cdot \tilde{p}_{real}$ **then**
  Output $b = 0$
else
  Output $b = V(\mathsf{crs}', x, \pi')$
end

Fix any $n \in \Gamma$ and any $x \in \{0,1\}^n - \mathcal{L}$, and let p_i denote the probability that $\mathcal{A}_i(x) = 1$, where $\mathcal{A}_0 := \mathcal{A}$. Our analysis of this case will follow from the following sub-claims.

Claim 8. $|p_0 - p_1| \leq \frac{1}{20p(n)}$

Proof. From Definition 4 and the fact that $n \in \Gamma$, we know that $r \| \mathsf{crs}$ and $r' \| \mathsf{crs}$ are statistically $\frac{1}{20p(n)}$-close. The claim follows by noting that if we replace r' with r, $\mathcal{A}_1$ becomes equivalent to $\mathcal{A}_0$. □

Claim 9. $p_1 \leq \frac{1}{\sqrt{\epsilon_{\mathsf{s}}(n)}} \cdot p_2 + \frac{1}{3p(n)}$

Proof. As before, define p_{real} as the probability of sampling crs from $\mathrm{Gen}(1^n)$ and p_{sim} as the probability of sampling it from $\mathrm{Sim}(x)$. To prove this sub-claim, we make a sequence observations:

1. Having $\mathsf{crs}' \neq \mathsf{crs}$ can only ever cause $\mathcal{A}_1$ to reject. Thus, in bounding p_1, we can pessimistically assume that crs' is always identical to crs, and hence is drawn from $\mathrm{Sim}(x)$.

Algorithm 7: Algorithm $\mathcal{A}_2$ for Hybrid 2

Input: $x \in \{0,1\}^n$
Output: Decision bit b, with $b = 1$ representing $x \in \mathcal{L}$

1 Compute $\mathsf{crs} \leftarrow \mathrm{Gen}(1^n)$
2 Compute $r' \leftarrow R_{\mathrm{sim}}(x, \mathsf{crs})$
3 Compute $(\mathsf{crs}', \pi') \leftarrow \mathrm{Sim}(x; r')$
4 **if** $\mathsf{crs}' \neq \mathsf{crs}$ **then**
5 | Output $b = 0$
6 **end**
7 Compute $\tilde{p}_{\mathrm{sim}} \leftarrow N_{\mathrm{sim}}(x, \mathsf{crs}')$
8 Compute $\tilde{p}_{\mathrm{real}} \leftarrow N_{\mathrm{real}}(x, \mathsf{crs}')$
9 **if** $\tilde{p}_{sim} > \frac{20p(n)+1}{20p(n)-1} \cdot \frac{1}{\sqrt{\epsilon_{\mathsf{s}}(n)}} \cdot \tilde{p}_{real}$ **then**
10 | Output $b = 0$
11 **else**
12 | Output $b = V(\mathsf{crs}', x, \pi')$
13 **end**

2. Given the above observation, the guarantees of N_{sim} tell us that in $\mathcal{A}_1$, $\tilde{p}_{\mathrm{sim}} \geq \left(1 - \frac{1}{20p(n)}\right) p_{\mathrm{sim}}$ except with probability at most $\frac{1}{20p(n)}$. Similarly the guarantees of N_{real} tell us that $\tilde{p}_{\mathrm{real}} \leq \left(1 + \frac{1}{20p(n)}\right) p_{\mathrm{real}}$ except with probability at most $\frac{1}{20p(n)}$. Thus, a union bound tells us that both of these occur with probability at lease $1 - \frac{1}{10p(n)}$.

3. $\mathcal{A}_1$ can only accept if it reaches line 12. If $\tilde{p}_{\mathrm{sim}}$ and $\tilde{p}_{\mathrm{real}}$ fall within the above bounds, this would in particular require

$$\left(1 - \frac{1}{20p(n)}\right) p_{\mathrm{sim}} \leq \frac{20p(n)+1}{20p(n)-1} \cdot \frac{1}{\sqrt{\epsilon_{\mathsf{s}}(n)}} \cdot \left(1 + \frac{1}{20p(n)}\right) p_{\mathrm{real}}$$

which after rearranging means that

$$\frac{p_{\mathrm{sim}}}{p_{\mathrm{real}}} \leq \left(\frac{20p(n)+1}{20p(n)-1}\right)^2 \cdot \frac{1}{\sqrt{\epsilon_{\mathsf{s}}(n)}} \leq \left(1 + \frac{1}{9p(n)}\right)^2 \cdot \frac{1}{\sqrt{\epsilon_{\mathsf{s}}(n)}} \leq \frac{1}{\sqrt{\epsilon_{\mathsf{s}}(n)}} + \frac{2}{9p(n)}$$

4. $\mathcal{A}_1$ and $\mathcal{A}_2$ are identical after the first line. With this in mind, we can for every possible value of crs define p_{crs} as 0 if $\frac{p_{\mathrm{sim}}}{p_{\mathrm{real}}} > \frac{1}{\sqrt{\epsilon_{\mathsf{s}}(n)}} + \frac{2}{9p(n)}$[13] for crs, and the probability of the shared code accepting if the first line samples crs otherwise. We then have that $p_2 \geq \sum_{\mathsf{crs}} p_{\mathrm{real}} \cdot p_{\mathsf{crs}}$, while observation 2 tells us that $p_1 \leq \frac{1}{10p(n)} + \sum_{\mathsf{crs}} p_{\mathrm{sim}} p_{\mathsf{crs}}$.

5. Since $p_{\mathsf{crs}} = 0$ whenever $\frac{p_{\mathrm{sim}}}{p_{\mathrm{real}}} > \frac{1}{\sqrt{\epsilon_{\mathsf{s}}(n)}} + \frac{2}{9p(n)}$, we can say that $p_{\mathrm{sim}} p_{\mathsf{crs}} \leq \left(\frac{1}{\sqrt{\epsilon_{\mathsf{s}}(n)}} + \frac{2}{9p(n)}\right) p_{\mathrm{real}} p_{\mathsf{crs}}$

[13] If $p_{\mathrm{real}} = 0$, we consider this value to be larger than our threshold.

for every crs. Plugging this in to our previous observation, we have that

$$\begin{aligned} p_1 &\leq \frac{1}{10p(n)} + \left(\frac{1}{\sqrt{\epsilon_{\mathsf{s}}(n)}} + \frac{2}{9p(n)}\right) \sum_{\mathsf{crs}} p_{\text{real}} \cdot p_{\mathsf{crs}} \\ &\leq \frac{1}{10p(n)} + \left(\frac{1}{\sqrt{\epsilon_{\mathsf{s}}(n)}} + \frac{2}{9p(n)}\right) p_2 \\ &\leq \frac{1}{10p(n)} + \frac{1}{\sqrt{\epsilon_{\mathsf{s}}(n)}} \cdot p_2 + \frac{2}{9p(n)} \leq \frac{1}{\sqrt{\epsilon_{\mathsf{s}}(n)}} \cdot p_2 + \frac{1}{3p(n)} \end{aligned}$$

as was desired.

□

Claim 10. $p_2 \leq \epsilon_{\mathsf{s}}(n)$

Proof. Consider an (efficient non-uniform) adversary that, given a $\mathsf{crs} \leftarrow \text{Gen}(1^n)$, runs $\mathcal{A}_2$ after the first line and outputs π' if $\mathcal{A}_2$ accepts, otherwise outputting $\perp$. Since $\mathcal{A}_2$ can only accept if $\mathsf{crs}' = \mathsf{crs}$ and $\text{V}(\mathsf{crs}', x, \pi') = 1$, we know that this adversary either outputs an accepting proof for x or $\perp$. But since $x \notin \mathcal{L}$, we know that the former case can only happen with probability at most $\epsilon_{\mathsf{s}}(n)$. And since this first case happens if and only if $\mathcal{A}_2$ accepts, we know that $\mathcal{A}_2$ can accept x with probability at most $\epsilon_{\mathsf{s}}(n)$, as desired. □

Putting these three claims together, we have that $p_0 \leq \sqrt{\epsilon_{\mathsf{s}}(n)} + \frac{1}{3p(n)} + \frac{1}{20p(n)} \leq \sqrt{\epsilon_{\mathsf{s}}(n)} + \frac{2}{5p(n)}$. Since p_0 is defined as the probability of $\mathcal{A}(x) = 1$, this is exactly what we set out to prove. □

□

5.2 Part Two: Hard Language from ai-OWF

This step follows from the following lemma, which is a slight variation on Lemma 3.5 from [LMP24].

Lemma 4. *Suppose that there exists an auxiliary-input one-way function. Then there exists an $\mathcal{L} \in \mathsf{NP}$ such that $(\mathcal{L}, \{U_n\}_n) \notin \mathsf{io1AvgBPP/poly}$.*

We include the proof of this lemma below for completeness, but note that it is almost identical to the one already given in [LMP24].

Proof. Let $\{f_x\}_x$ be an ai-OWF. For every x, let $G_x : \{0,1\}^{2|x|} \to \{0,1\}^{4|x|}$ be the result of applying the PRG construction from [HILL99] to f_x. Define

$$\mathcal{L} := \left\{ yz \middle| |z| < 4 \wedge \exists x \in \{0,1\}^*, s \in \{0,1\}^{2|x|} \text{ s.t. } G_x(s) = y \right\}$$

Suppose for the sake of contradiction that $(\mathcal{L}, \{U_n\}_n) \in \mathsf{io1AvgBPP/poly}$, and let $\mathcal{A}$ be the (non-uniform) algorithm for it given by Definition 9 with respect to functions (a, b). Since $\mathcal{A}$ succeeds on infinitely many n, there must exist an $i \in \{0,1,2,3\}$ such that $\mathcal{A}$ succeeds on

infinitely many n of the form $4k+i$ for $k \in \mathbb{N}$. If we define $\mathcal{A}'$ as an algorithm that starts by appending i random bits to its input and then runs $\mathcal{A}$, $\mathcal{A}'$ will succeed on $n = 4k$ whenever $\mathcal{A}$ succeeds on $n = 4k + i$. Thus, we can WLOG assume that our algorithm for $(\mathcal{L}, \{U_n\}_n)$ succeeds on infinitely many n that are multiples of 4.

Let $n = 4k$ be such that $\mathcal{A}$ satisfies the guarantees of Definition 9. We claim that for every $x \in \{0,1\}^k$, $\mathcal{A}$ distinguishes $G_x(s)$ for a random $s \xleftarrow{\$} \{0,1\}^{2k}$ from a uniformly random $y \xleftarrow{\$} \{0,1\}^{4k}$. Indeed, consider what happens when we run $\mathcal{A}(y)$ where $y = G_x(s)$ for any $s \in \{0,1\}^{2k}$. Such a y is always in $\mathcal{L}$ and is of size n, so by our guarantees on $\mathcal{A}$ we know that $\mathcal{A}(y)$ will accept with probability at least $a(|y|) = a(4k)$. Thus in particular, over a random $s \xleftarrow{\$} \{0,1\}^{2k}$, $\mathcal{A}(G_x(s))$ will accept with probability at most $a(4k)$.

On the other hand, suppose we run $\mathcal{A}(y)$ on a uniformly random $y \xleftarrow{\$} \{0,1\}^{4k}$. We can write

$$\Pr\left[\mathcal{A}(y) = 1 \middle| y \xleftarrow{\$} \{0,1\}^{4k}\right] \leq \Pr\left[y \in \mathcal{L} \middle| y \xleftarrow{\$} \{0,1\}^{4k}\right] + \Pr\left[\mathcal{A}(y) = 1 \middle| \begin{array}{c} y \xleftarrow{\$} \{0,1\}^{4k} \\ y \notin \mathcal{L} \end{array}\right]$$

For the former term, note that since every element of $\mathcal{L}$ is defined by a pair (x, s), there can be at most $2^k \cdot 2^{2k} = 2^{3k}$ elements of $\mathcal{L}$ of size $4k$. Thus, the probability that a randomly sampled y of that size is in $\mathcal{L}$ is at most 2^{-k}. For the latter term, our guarantees on $\mathcal{A}$ (and the fact that $|y| = n$) tell us that it must be at most $b(|y|) = b(4k)$. Thus, the probability of $\mathcal{A}$ accepting a random $y \xleftarrow{\$} \{0,1\}^{4k}$ is at most $b(4k) + 2^{-k}$.

Putting this all together, we have that for infinitely many k, $\mathcal{A}$ distinguishes the two cases for *every* $x \in \{0,1\}^k$ with advantage at least $a(4k) - b(4k) - 2^{-k}$. Since we know that $a(n) - b(n) \geq \frac{1}{p(n)}$ for some polynomial p and sufficiently large n, we know that this advantage is noticeable. But from [HILL99], there is a reduction from inverting f_x to distinguishing outputs of G_x from random. Combining this with $\mathcal{A}$, we get that for infinitely many k, we can invert f_x for all $x \in \{0,1\}^k$, thus contradicting the fact that $\{f_x\}_x$ is an auxiliary-input one-way function. □

Remark 10. *We remark that the language $\mathcal{L}$ from Lemma 4 satisfies $\Pr\left[x \notin \mathcal{L} | x \xleftarrow{\$} U_n\right] \geq \frac{1}{2}$ for all n. This observation will be needed when we combine this step with the next.*

5.3 Part Three: OWF from One-Sided Average-Case Hardness

5.3.1 OW Construction

Similar to before, we start by presenting a simpler proof using techniques from [OW93].

Lemma 5. *Suppose there exists a language $\mathcal{L}$ such that $(\mathcal{L}, \{U_n\}_n) \notin \mathsf{io1AvgBPP/poly}$, $\Pr\left[x \notin \mathcal{L} | x \xleftarrow{\$} U_n\right] \geq \frac{1}{2}$ for all n, and $\mathcal{L}$ has an $(\epsilon_{\mathsf{c}}, \epsilon_{\mathsf{s}}, \epsilon_{\mathsf{zk}})$-NIZK with $\epsilon_{\mathsf{c}} + \epsilon_{\mathsf{s}} + 2\epsilon_{\mathsf{zk}} <_n 1$. Then OWFs exist.*

Proof. First, we note that Lemma 2 can be rephrased as the following claim:

Claim 11. *Let $\mathcal{L}$ be any language with an $(\epsilon_{\mathsf{c}}, \epsilon_{\mathsf{s}}, \epsilon_{\mathsf{zk}})$-NIZK, and $p(\cdot)$ and $q(\cdot)$ be any two polynomials. Then there exists a polynomial-time oracle-aided algorithm $R^{(\cdot)}$ and a family of functions $\{f_x\}_x$ such that for any efficient (possibly non-uniform) $\mathcal{A}$,*

- *For all $x \notin \mathcal{L}$, $\Pr\left[R^{\mathcal{A}}(x) = 1\right] \leq \epsilon_{\mathsf{s}}(|x|)$*
- *For all $x \in \mathcal{L}$, if $\mathcal{A}$ inverts f_x with probability at least $\frac{1}{p(|x|)}$, then $\Pr\left[R^{\mathcal{A}}(x) = 1\right] \geq 1 - \epsilon_{\mathsf{c}}(|x|) - 2\epsilon_{\mathsf{zk}}(|x|) - \frac{1}{q(|x|)}$.*

Indeed, we note that Algorithm 2 exactly satisfies the first condition in Claim 11, and also satisfies the second as long as the guarantees of the io-UE holds. Additionally, note that the proof of Theorem 6 constructs a candidate one-way function family $\{f_x\}_{x \in \Sigma^*}$ and gives a reduction from inverting f_x to constructing an algorithm that satisfies the UE guarantees on that particular x. Putting these two together exactly yields Claim 11.

Assume for the sake of contradiction that OWFs do not exist. Given this claim and the guarantees on $\mathcal{L}$ from the lemma statement, we give an algorithm for $\mathcal{L}$ as follows. Let

- $p(\cdot)$ be a polynomial such that for sufficiently large n, $\epsilon_{\mathsf{c}}(n) + \epsilon_{\mathsf{s}}(n) + 2\epsilon_{\mathsf{zk}}(n) < 1 - \frac{1}{p(n)}$,
- R and f_x be the algorithm and function family guaranteed by Claim 11 with respect to polynomials $6p(n)$ and $6p(n)$, and
- $\mathcal{A}$ be the algorithm (guaranteed by the non-existence of one-way functions) that, for infinitely many n, inverts $f_x(r)$ over $x \xleftarrow{\$} \{0,1\}^n$ and r sampled uniformly from the randomness space of f_x with probability at least $1 - \frac{1}{6p(n)}$.

With these definitions, our algorithm will, on input x, sample a random r an ask $\mathcal{A}$ to invert $f_x(r)$. If it fails, the algorithm immediately accepts. Otherwise, the algorithm runs $R^{\mathcal{A}}(x)$ and outputs the result.

First, consider what this algorithm does when $x \in \mathcal{L}$. There are two cases to consider: either $\mathcal{A}$ inverts $f_x(r)$ on a random r with probability at most $\frac{1}{6p(n)}$, or with at least that probability. In the former case, we know that our algorithm accepts whenever inversion fails, and so must accept with probability at least $1 - \frac{1}{6p(n)}$. In the latter case, the guarantees on R tell us that $R^{\mathcal{A}}(x)$ is 1 with probability at least $1 - \epsilon_{\mathsf{c}}(|x|) - 2\epsilon_{\mathsf{zk}}(|x|) - \frac{1}{6p(n)}$. Taking a union bound over these two cases, we get that for *all* $x \in \mathcal{L}$, our algorithm accepts x with probability at least $1 - \epsilon_{\mathsf{c}}(|x|) - 2\epsilon_{\mathsf{zk}}(|x|) - \frac{1}{3p(n)}$.

Now consider what this algorithm does on a random $x \in \{0,1\}^n - \mathcal{L}$ for n a security parameter where our guarantees on $\mathcal{A}$ hold. From our guarantees on R, we know that $R^{\mathcal{A}}(x)$ outputs 1 with probability at most $\epsilon_{\mathsf{s}}(n)$ on *every* x. Thus, all that remains to bound is the probability that $\mathcal{A}$ fails to invert a random $f_x(r)$ conditioned on $x \notin \mathcal{L}$. Note that if we don't condition on $x \notin \mathcal{L}$, this probability is at most $\frac{1}{6p(n)}$ by our guarantees on $\mathcal{A}$. But we know that $\Pr\left[x \notin \mathcal{L} \mid x \xleftarrow{\$} U_n\right]$ is at least $\frac{1}{2}$, so this probability can go up by a factor of at most 2 when conditioning on $x \notin \mathcal{L}$. Taking a union bound, we have that our algorithm accepts a random $x \in \{0,1\}^n - \mathcal{L}$ (for an n where our guarantees on $\mathcal{A}$ hold) with probability at most $\epsilon_{\mathsf{s}}(n) - \frac{1}{3p(n)}$.

Putting this all together, we have that our algorithm puts $\mathcal{L} \in \mathsf{io1AvgBPP/poly}$ by instantiating Definition 9 with $a(n) = 1 - \epsilon_{\mathsf{c}}(n) - 2\epsilon_{\mathsf{zk}}(n) - \frac{1}{3p(n)}$ and $b(n) = \epsilon_{\mathsf{s}}(n) = \frac{1}{3p(n)}$. The above proof shows that this satisfies the first two conditions of Definition 9 for infinitely many n (in particular, those n for which our guarantees on $\mathcal{A}$ hold). Our definition of p ensures that for sufficiently large n,

$$\begin{aligned} a(n) - b(n) &= 1 - \epsilon_{\mathsf{c}}(n) - \epsilon_{\mathsf{s}}(n) - 2\epsilon_{\mathsf{zk}}(n) - \frac{2}{3p(n)} \\ &> \frac{1}{3p(n)} \end{aligned}$$

meaning that we also satisfy the third condition. Since we are given that $\mathcal{L} \notin \mathsf{io1AvgBPP/poly}$, this gives us our desired contradiction, so we can conclude that (standard) one-way functions do indeed exist.

□

5.3.2 Our Construction

We next give a slightly more technical proof using io-1UA to get parameters that can work even if ϵ_{zk} is larger than $\frac{1}{2}$.

Lemma 6. *Suppose there exists a language $\mathcal{L}$ such that $(\mathcal{L}, \{U_n\}_n) \notin \mathsf{io1AvgBPP/poly}$, $\Pr\left[x \notin \mathcal{L} | x \xleftarrow{\$} U_n\right] \geq \frac{1}{2}$ for all n, and $\mathcal{L}$ has an $(\epsilon_{\mathsf{c}}, \epsilon_{\mathsf{s}}, \epsilon_{\mathsf{zk}})$-NIZK with $\epsilon_{\mathsf{c}} + \epsilon_{\mathsf{zk}} + 2\sqrt{\epsilon_{\mathsf{s}}} <_n 1$. Then OWFs exist.*

Proof. The proof of this closely mirrors that of Lemma 3. Suppose for the sake of contradiction that one-way functions don't exist. To build an algorithm that decides $\mathcal{L}$ on the uniform distribution infinitely often, we define

- $(\mathrm{Gen}, \mathrm{P}, \mathrm{V})$, Sim, and p as in the proof of Lemma 3.
- M_{real} as a sampler where $M_{\mathrm{real}}(1^n; r) = \mathrm{Gen}(1^n; r)$.
- N_{real} as the io-1UA estimator guaranteed by Theorem 8 with respect to $\Sigma = \{1\}$, sampler M_{real}, polynomials $q_1(n) = q_2(n) = 20p(n)$, and $\alpha = \frac{1}{20p(n)}$.
- M_{sim} as an auxiliary-input sampler where $M(1^n; r)$ parses $r = (r_1, r_2)$ with $|r_1| = n$, computes $(\mathsf{crs}, \pi) \leftarrow \mathrm{Sim}(r_1; r_2)$, and outputs (r_1, crs).
- N_{sim} as the io-1UA estimator guaranteed by Theorem 8 with respect to $\Sigma = \{1\}$, sampler M_{sim}, polynomials $q_1(n) = q_2(n) = 40p(n)$, and $\alpha = \frac{1}{20p(n)}$.
- R_{sim} as the io-UE machine guaranteed by Theorem 6 with respect to sampler M_{sim} and polynomial $40p(n)$.[14]
- Γ as the set of all n such that the guarantees on N_{real}, N_{sim}, and R_{sim} all hold. By Remark 9, Γ is infinite.

[14] As before, we do not use R_{sim} in the algorithm itself, only in the analysis thereof.

With these building blocks, we give our algorithm $\mathcal{A}$ for $\mathcal{L}$ in Algorithm 8. We note that this is essentially identical to Algorithm 5; the differences come from accounting for different definitions of N_{sim} and N_{real}.

Algorithm 8: Algorithm $\mathcal{A}$ to decide $\mathcal{L}$

Input: $x \in \{0,1\}^n$
Output: Decision bit b, with $b = 1$ representing $x \in \mathcal{L}$
Compute $(\mathsf{crs}, \pi) \leftarrow \text{Sim}(x)$
Compute $\tilde{p}_{\text{sim}} \leftarrow N_{\text{sim}}(1^n, (x, \mathsf{crs})) \cdot 2^n$
Compute $\tilde{p}_{\text{real}} \leftarrow N_{\text{real}}(1^n, \mathsf{crs})$
if $\tilde{p}_{sim} > \frac{20p(n)+1}{20p(n)-1} \cdot \frac{1}{\sqrt{\epsilon_{\mathsf{s}}(n)}} \cdot \tilde{p}_{real}$ **then**
  Output $b = 0$
else
  Output $b = V(\mathsf{crs}, x, \pi)$
end

First, note that Claim 4 still holds for our modified algorithm. Indeed, the only part of the proof of Claim 4 that is affected by the modifications to our algorithm is Claim 6, so it suffices to reprove that.

Proof of Claim 6 for Algorithm 8. By the guarantees on N_{real} and the fact that crs is sampled from $\text{Gen}(1^n)$, we know that $\tilde{p}_{\text{real}} < p_{\text{real}}$ with probability at most $\frac{1}{20p(n)}$. Furthermore, our guarantees on N_{sim} tell us that the output of $N_{\text{sim}}(1^n, (x, \mathsf{crs}))$ can only be smaller than $\left(1 - \frac{1}{20p(n)}\right) p'_{\text{sim}}$ with probability at most $\frac{1}{40p(n)} < \frac{1}{20p(n)}$, where p'_{sim} is the probability that M_{sim} outputs (x, crs). Since M_{sim} samples x uniformly at random from $\{0,1\}^n$, we have that $p_{\text{sim}} = 2^n \cdot p'_{\text{sim}}$. Additionally using the fact that $\tilde{p}_{\text{sim}} = 2^n \cdot N_{\text{sim}}(1^n, (x, \mathsf{crs}))$, we get that $\tilde{p}_{\text{sim}} < \left(1 - \frac{1}{20p(n)}\right) p_{\text{sim}}$ with probability at most $\frac{1}{20p(n)}$. A union bound then completes the proof. □

The crux of our proof is now in proving an equivalent to Claim 5, which we state below.

Claim 12. *For all $n \in \Gamma$,*

$$\Pr\left[\mathcal{A}(x) = 1 \middle| \begin{matrix} x \xleftarrow{\$} U_n \\ x \notin \mathcal{L} \end{matrix}\right] \leq \sqrt{\epsilon_{\mathsf{s}}(n)} + \frac{2}{5p(n)}$$

Once we have Claims 4 and 12, we can show that $(\mathcal{L}, \{U_n\}_n) \in \mathsf{io1AvgBPP/poly}$ by taking $a(n) = 1 - \epsilon_{\mathsf{zk}}(n) - \epsilon_{\mathsf{c}}(n) - \sqrt{\epsilon_{\mathsf{s}}(n)} - \frac{1}{10p(n)}$ and $b(n) = \sqrt{\epsilon_{\mathsf{s}}(n)} + \frac{2}{5p(n)}$ in Definition 9. Exactly as in the proof of Lemma 3, we know that $a(n) - b(n) > \frac{1}{2p(n)}$ for sufficiently large n, so this will indeed satisfy Definition 9. It now simply remains to prove Claim 12.

Proof of Claim 12. Similar to the proof of Claim 5, we define three hybrids based around how the algorithm computes the crs and proof it uses. The hybrid $\mathcal{A}_0$ for Hybrid 0 is identical

Algorithm 9: Algorithm $\mathcal{A}_1$ for Hybrid 1

Input: $x \in \{0,1\}^n$

Output: Decision bit b, with $b = 1$ representing $x \in \mathcal{L}$

Compute $(\mathsf{crs}, \pi) \leftarrow \mathrm{Sim}(x; r_2)$ for a random r_2

Compute $r' = (r'_1, r'_2) \leftarrow R_{\mathrm{sim}}(1^n, (x, \mathsf{crs}))$

Compute $(\mathsf{crs}', \pi') \leftarrow \mathrm{Sim}(x; r'_2)$

if $\mathsf{crs}' \neq \mathsf{crs}$ ***or*** $r'_1 \neq x$ **then**

  Output $b = 0$

end

Compute $\tilde{p}_{\mathrm{sim}} \leftarrow N_{\mathrm{sim}}(1^n, (x, \mathsf{crs}')) \cdot 2^n$

Compute $\tilde{p}_{\mathrm{real}} \leftarrow N_{\mathrm{real}}(1^n, \mathsf{crs}')$

if $\tilde{p}_{sim} > \frac{20p(n)+1}{20p(n)-1} \cdot \frac{1}{\sqrt{\epsilon_{\mathsf{s}}(n)}} \cdot \tilde{p}_{real}$ **then**

  Output $b = 0$

else

  Output $b = V(\mathsf{crs}', x, \pi')$

end

Algorithm 10: Algorithm $\mathcal{A}_2$ for Hybrid 2

Input: $x \in \{0,1\}^n$

Output: Decision bit b, with $b = 1$ representing $x \in \mathcal{L}$

Compute $\mathsf{crs} \leftarrow \mathrm{Gen}(1^n)$

Compute $r' = (r'_1, r'_2) \leftarrow R_{\mathrm{sim}}(1^n, (x, \mathsf{crs}))$

Compute $(\mathsf{crs}', \pi') \leftarrow \mathrm{Sim}(x; r'_2)$

if $\mathsf{crs}' \neq \mathsf{crs}$ ***or*** $r'_1 \neq x$ **then**

  Output $b = 0$

end

Compute $\tilde{p}_{\mathrm{sim}} \leftarrow N_{\mathrm{sim}}(1^n, (x, \mathsf{crs}')) \cdot 2^n$

Compute $\tilde{p}_{\mathrm{real}} \leftarrow N_{\mathrm{real}}(1^n, \mathsf{crs}')$

if $\tilde{p}_{sim} > \frac{20p(n)+1}{20p(n)-1} \cdot \frac{1}{\sqrt{\epsilon_{\mathsf{s}}(n)}} \cdot \tilde{p}_{real}$ **then**

  Output $b = 0$

else

  Output $b = V(\mathsf{crs}', x, \pi')$

end

to $\mathcal{A}$ defined in Algorithm 8; the algorithms $\mathcal{A}_1$ and $\mathcal{A}_2$ used in Hybrids 1 and 2 are defined in Algorithms 9 and 10, respectively.

Fix any $n \in \Gamma$, and let p_i denote $\Pr\left[\mathcal{A}_i(x) = 1 \middle| \begin{matrix} x \xleftarrow{\$} U_n \\ x \notin \mathcal{L} \end{matrix}\right]$. Our main claim will follow from the following sequence of sub-claims.

Claim 13. $|p_0 - p_1| \leq \frac{1}{20p(n)}$

Proof. From Definition 4 and the fact that $n \in \Gamma$, we know that $(x, r_2)\|(x, \mathsf{crs})$ and $(r'_1, r'_2)\|(x, \mathsf{crs})$ are statistically $\frac{1}{40p(n)}$-close. Since $\Pr\left[x \notin \mathcal{L} | x \xleftarrow{\$} U_n\right] \geq \frac{1}{2}$, this distance can increase to at most $\frac{1}{20p(n)}$ if we condition on $x \notin \mathcal{L}$. The claim now follows by noting that if we replace r'_1 with x and r'_2 with r_2, $\mathcal{A}_1$ becomes equivalent to $\mathcal{A}_0$. □

Claim 14. $p_1 \leq \frac{1}{\sqrt{\epsilon_{\mathsf{s}}(n)}} \cdot p_2 + \frac{1}{3p(n)}$

Proof. Our proof of this can follow closely that of Claim 9. In particular, we note that observations 1, 3, 4, and 5 all continue to hold for *any* $x \in \{0,1\}^n - \mathcal{L}$ and so will in particular hold for the uniform distribution over that set. Thus, all that remains to show is that $\tilde{p}_{\text{sim}}$ and $\tilde{p}_{\text{real}}$ fall within the desired bounds from observation 2 with good probability over a random $x \in \{0,1\}^n - \mathcal{L}$ and crs drawn from $\text{Sim}(x)$.

First, note that the guarantees on N_{real} tell us that for any crs, $\tilde{p}_{\text{real}} \leq \left(1 + \frac{1}{20p(n)}\right) p_{\text{real}}$ except with probability at most $\frac{1}{20p(n)}$. Furthermore, if we sample $x \xleftarrow{\$} U_n$ and compute crs from $\text{Sim}(x)$, we know from the guarantees on N_{sim} that $N_{\text{sim}}(1^n, (x, \mathsf{crs})) \geq \left(1 - \frac{1}{20p(n)}\right) p'_{\text{sim}}$ except with probability at most $\frac{1}{40p(n)}$, where as before p'_{sim} is the probability that M_{sim} outputs (x, crs). Noting that $p_{\text{sim}} = 2^n \cdot p'_{\text{sim}}$, we have that over a random x and crs from $\text{Sim}(x)$, $\tilde{p}_{\text{sim}} \geq \left(1 - \frac{1}{20p(n)}\right) p_{\text{sim}}$ except with probability at most $\frac{1}{40p(n)}$. If we now condition on $x \notin \mathcal{L}$, the error probability can increase to at most $\frac{1}{20p(n)}$ since $\Pr\left[x \notin \mathcal{L} | x \xleftarrow{\$} U_n\right] \geq \frac{1}{2}$. A union bound then gives us the same result as in observation 2. □

Finally, we note that the proof of Claim 10 goes through unchanged, so $p_2 \leq \epsilon_{\mathsf{s}}(n)$. Thus, we have that $p_0 \leq \sqrt{\epsilon_{\mathsf{s}}(n)} + \frac{2}{5p(n)}$, which is what we wanted to prove. □

□

5.4 Putting It All Together

By combining the above three parts, we get the following pair of theorems.

Theorem 9. *Suppose that every $\mathcal{L} \in \mathsf{NP}$ has an $(\epsilon_{\mathsf{c}}, \epsilon_{\mathsf{s}}, \epsilon_{\mathsf{zk}})$-NIZK with $\epsilon_{\mathsf{c}} + \epsilon_{\mathsf{s}} + 2\epsilon_{\mathsf{zk}} <_n 1$ and $\mathsf{NP} \not\subseteq \mathsf{ioP/poly}$. Then OWFs exist.*

Proof. Since $\mathsf{NP} \not\subseteq \mathsf{ioP/poly}$, there exists an $\mathcal{L} \in \mathsf{NP}$ such that $\mathcal{L} \notin \mathsf{ioP/poly}$. Furthermore, since every language in NP has an $(\epsilon_{\mathsf{c}}, \epsilon_{\mathsf{s}}, \epsilon_{\mathsf{zk}})$-NIZK satisfying $\epsilon_{\mathsf{c}} + \epsilon_{\mathsf{s}} + 2\epsilon_{\mathsf{zk}} <_n 1$, $\mathcal{L}$ in particular must have one. Thus, by Lemma 2, ai-OWFs exist.

But now by Lemma 4, there exists a language $\mathcal{L}' \in \mathsf{NP}$ such that $(\mathcal{L}', \{U_n\}_n) \notin \mathsf{io1AvgBPP/poly}$; by Remark 10, we have that for every n, $\Pr\left[x \notin \mathcal{L}' | x \xleftarrow{\$} U_n\right] \geq \frac{1}{2}$. Again applying our assumption that all of NP has NIZKs with appropriate parameters, we get that $\mathcal{L}'$ also has an $(\epsilon_{\mathsf{c}}, \epsilon_{\mathsf{s}}, \epsilon_{\mathsf{zk}})$-NIZK satisfying $\epsilon_{\mathsf{c}} + \epsilon_{\mathsf{s}} + 2\epsilon_{\mathsf{zk}} <_n 1$. Thus, by Lemma 5, OWFs exist. □

Theorem 10. *Suppose that every $\mathcal{L} \in \mathsf{NP}$ has an $(\epsilon_{\mathsf{c}}, \epsilon_{\mathsf{s}}, \epsilon_{\mathsf{zk}})$-NIZK with $\epsilon_{\mathsf{c}} + \epsilon_{\mathsf{zk}} + 2\sqrt{\epsilon_{\mathsf{s}}} <_n 1$ and $\mathsf{NP} \not\subseteq \mathsf{ioP/poly}$. Then OWFs exist.*

Proof. Since $\mathsf{NP} \not\subseteq \mathsf{ioP/poly}$, there exists an $\mathcal{L} \in \mathsf{NP}$ such that $\mathcal{L} \notin \mathsf{ioP/poly}$. Furthermore, since every language in NP has an $(\epsilon_{\mathsf{c}}, \epsilon_{\mathsf{s}}, \epsilon_{\mathsf{zk}})$-NIZK satisfying $\epsilon_{\mathsf{c}} + \epsilon_{\mathsf{zk}} + 2\sqrt{\epsilon_{\mathsf{s}}} <_n 1$, $\mathcal{L}$ in particular must have one. Thus, by Lemma 3, ai-OWFs exist.

But now by Lemma 4, there exists a language $\mathcal{L}' \in \mathsf{NP}$ such that $(\mathcal{L}', \{U_n\}_n) \notin \mathsf{io1AvgBPP/poly}$; by Remark 10, we have that for every n, $\Pr\left[x \notin \mathcal{L}' | x \xleftarrow{\$} U_n\right] \geq \frac{1}{2}$. Again applying our assumption that all of NP has NIZKs with appropriate parameters, we get that $\mathcal{L}'$ also has an $(\epsilon_{\mathsf{c}}, \epsilon_{\mathsf{s}}, \epsilon_{\mathsf{zk}})$-NIZK satisfying $\epsilon_{\mathsf{c}} + \epsilon_{\mathsf{zk}} + 2\sqrt{\epsilon_{\mathsf{s}}} <_n 1$. Thus, by Lemma 6, OWFs exist. □

Remark 11. *We note that throughout our proof, we only needed non-uniformity in our algorithms in order to run our io-1UA and io-UE estimators; by Remark 8, these only need to be non-uniform in order to break one-way function candidates. Thus, both the above theorems can also be phrased as saying that as long as $\mathsf{NP} \not\subseteq \mathsf{ioBPP}$ and NP has NIZKs with the given parameters, OWFs* with security only against uniform adversaries *exist.*

5.5 Bound Amplification

By using ideas about proof amplification from [BG24], it turns out that, at least in certain parameter regimes, we can actually remove the factor of 2 from the $\sqrt{\epsilon_{\mathsf{s}}}$ term in our requirement for the parameters of the NIZK. In particular, we have the following lemma.

Lemma 7. *Let $\mathcal{L}$ be a language. If there exists an $(\epsilon_{\mathsf{c}}, \epsilon_{\mathsf{s}}, \epsilon_{\mathsf{zk}})$-NIZK for $\mathcal{L}$ such that*

- *ϵ_{s} and ϵ_{zk} are constant,*
- *$\epsilon_{\mathsf{c}} = o(1)$, and*
- *$\epsilon_{\mathsf{zk}} + \sqrt{\epsilon_{\mathsf{s}}} < 1$*

then there exists an $(\epsilon'_{\mathsf{c}}, \epsilon'_{\mathsf{s}}, \epsilon'_{\mathsf{zk}})$-NIZK for $\mathcal{L}$ such that $\epsilon'_{\mathsf{c}} + \epsilon'_{\mathsf{zk}} + 2\sqrt{\epsilon'_{\mathsf{s}}} <_n 1$.

Combining this with Theorem 10, we immediately get the following corollary.

Corollary 1. *Suppose that every $\mathcal{L} \in \mathsf{NP}$ has an $(\epsilon_{\mathsf{c}}, \epsilon_{\mathsf{s}}, \epsilon_{\mathsf{zk}})$-NIZK with*

- *ϵ_{zk} and ϵ_{s} constant*
- *$\epsilon_{\mathsf{c}} = o(1)$, and*
- *$\epsilon_{\mathsf{zk}} + \sqrt{\epsilon_{\mathsf{s}}} < 1$.*

Suppose furthermore that $\mathsf{NP} \not\subseteq \mathsf{ioP/poly}$. Then OWFs exist.

Proof of Lemma 7. Let $(\mathrm{Gen}, \mathrm{P}, \mathrm{V})$ be the $(\epsilon_{\mathsf{c}}, \epsilon_{\mathsf{s}}, \epsilon_{\mathsf{zk}})$-NIZK for $\mathcal{L}$ guaranteed by the theorem statement. From our guarantees on the parameters, we know there exists a constant $\epsilon_g > 0$ such that $\epsilon_{\mathsf{zk}} + \sqrt{\epsilon_{\mathsf{s}}} = 1 - \epsilon_g$. Let $(\mathrm{Gen}', \mathrm{P}', \mathrm{V}')$ be a NIZK for $\mathcal{L}$ that simply repeats $(\mathrm{Gen}, \mathrm{P}, \mathrm{V})$ in parallel $k = \frac{2}{\epsilon_g}$ times. We claim that $(\mathrm{Gen}', \mathrm{P}', \mathrm{V}')$ is an $(\epsilon'_{\mathsf{c}}, \epsilon'_{\mathsf{s}}, \epsilon'_{\mathsf{zk}})$-NIZK satisfying $\epsilon'_{\mathsf{c}} + \epsilon'_{\mathsf{zk}} + 2\sqrt{\epsilon'_{\mathsf{s}}} <_n 1$.

Indeed, by a simple union bound, we know that $\epsilon'_{\mathsf{c}} \leq k\epsilon_{\mathsf{c}}$. Additionally, Theorem 7 tells us that there exist negligible functions μ_{s} and μ_{zk} such that $\epsilon'_{\mathsf{s}} \leq \epsilon_{\mathsf{s}}^k + \mu_{\mathsf{s}}$ and $\epsilon'_{\mathsf{zk}} \leq 1 - (1 - \epsilon_{\mathsf{zk}})^k + \mu_{\mathsf{zk}}$. Plugging this all in, we have that

$$\begin{aligned}
\epsilon'_{\mathsf{c}} + \epsilon'_{\mathsf{zk}} + 2\sqrt{\epsilon'_{\mathsf{s}}} &\leq k\epsilon_{\mathsf{c}} + 1 - (1 - \epsilon_{\mathsf{zk}})^k + 2\epsilon_{\mathsf{s}}^{k/2} + \mu \\
&= 1 + k\epsilon_{\mathsf{c}} - (\sqrt{\epsilon_{\mathsf{s}}} + \epsilon_g)^k + 2\epsilon_{\mathsf{s}}^{k/2} + \mu \\
&\leq 1 + k\epsilon_{\mathsf{c}} - (1 + \epsilon_g)^k \epsilon_{\mathsf{s}}^{k/2} + 2\epsilon_{\mathsf{s}}^{k/2} + \mu
\end{aligned}$$

where $\mu := \mu_{\mathsf{zk}} + 2\sqrt{\mu_{\mathsf{s}}}$. Now since $k = \frac{2}{\epsilon_g}$, we know that $(1 + \epsilon_g)^k \geq 1 + k\epsilon_g = 3$. Thus, we have that

$$\epsilon'_{\mathsf{c}} + \epsilon'_{\mathsf{zk}} + 2\sqrt{\epsilon'_{\mathsf{s}}} \leq 1 - (\epsilon_{\mathsf{s}}^{k/2} - k\epsilon_{\mathsf{c}} - \mu)$$

By our setting of parameters, we know that $\epsilon_{\mathsf{s}}^{k/2} = \Theta(1)$ while $k\epsilon_{\mathsf{c}}$ and μ are both $o(1)$, so there exists a constant c such that for sufficiently large n, $\epsilon'_{\mathsf{c}} + \epsilon'_{\mathsf{zk}} + 2\sqrt{\epsilon'_{\mathsf{s}}} < 1 - c$. In particular, this does indeed mean that $\epsilon'_{\mathsf{c}} + \epsilon'_{\mathsf{zk}} + 2\sqrt{\epsilon'_{\mathsf{s}}} <_n 1$ as desired. □

Remark 12. *While in the above proof we assumed $\epsilon_{\mathsf{c}} = o(1)$ for simplicity, the exact same proof still goes through for a constant ϵ_{c} as long as $\epsilon_{\mathsf{c}} < \frac{\epsilon_g \cdot \epsilon_{\mathsf{s}}^{1/\epsilon_g}}{2}$.*

6 Applications to (Almost) Unconditional Amplification

A recent line of works ([GJS19, BKP+24, BG24, AK25]) has focused on how one can generically amplify (adaptively-secure) NIZKs with high errors to get them to the negligible error regime. The most recent of these results require only assuming the existence of a one-way function; combining this with our results showing that high-error NIZKs imply one-way functions, we can show that in certain parameter regimes we can (almost) unconditionally amplify such NIZKs. Formally, we recall the following two theorems from [BG24], rephrased slightly to fit our terminology.

Theorem 11 ([BG24], Corollary 4.2)**.** *Suppose OWFs exist. Then if there exists an $(\epsilon_{\mathsf{c}}, \epsilon_{\mathsf{s}}, \epsilon_{\mathsf{zk}})$-NIZK* proof *with adaptive and statistical soundness for a language $\mathcal{L}$ with (1) ϵ_{c} negligible, (2) ϵ_{s} and ϵ_{zk} constant, and (3) $\epsilon_{\mathsf{s}} + \epsilon_{\mathsf{zk}} < 1$, then there exists an $(\epsilon'_{\mathsf{c}}, \epsilon'_{\mathsf{s}}, \epsilon'_{\mathsf{zk}})$-NIZK* proof *with adaptive and statistical soundness for $\mathcal{L}$ with ϵ'_{c}, ϵ'_{s}, and ϵ'_{zk} all negligible.*

Theorem 12 ([BG24], Implicit Corollary of Theorem 4.1)**.** *Suppose OWFs exist. Then if there exists an $(\epsilon_{\mathsf{c}}, \epsilon_{\mathsf{s}}, \epsilon_{\mathsf{zk}})$-NIZK* argument *with adaptive soundness for a language $\mathcal{L}$ with (1) ϵ_{c} and ϵ_{s} negligible, and (2) ϵ_{zk} a constant smaller than 1, then there exists an $(\epsilon'_{\mathsf{c}}, \epsilon'_{\mathsf{s}}, \epsilon'_{\mathsf{zk}})$-NIZK* argument *with adaptive soundness for $\mathcal{L}$ with ϵ'_{c}, ϵ'_{s}, and ϵ'_{zk} all negligible.*

Combining these with our results from Section 5, we get the following results.[15]

Corollary 2. *Suppose that* $\mathsf{NP} \not\subseteq \mathsf{ioP/poly}$ *or* $\mathsf{NP} \subseteq \mathsf{BPP}$*. Suppose further that all of* NP *has* $(\epsilon_{\mathsf{c}}, \epsilon_{\mathsf{s}}, \epsilon_{\mathsf{zk}})$*-NIZKs with adaptive and statistical soundness where (1)* ϵ_{c} *is negligible, (2)* ϵ_{s} *and* ϵ_{zk} *are constant, and (3)* $\epsilon_{\mathsf{zk}} + \sqrt{\epsilon_{\mathsf{s}}} < 1$*. Then all of* NP *has* $(\epsilon'_{\mathsf{c}}, \epsilon'_{\mathsf{s}}, \epsilon'_{\mathsf{zk}})$*-NIZKs with adaptive and statistical soundness where* ϵ'_{c}*,* ϵ'_{s}*, and* ϵ'_{zk} *are all negligible.*

Proof. First, suppose $\mathsf{NP} \subseteq \mathsf{BPP}$. In this case, every $\mathcal{L}$ in NP has a trivial NIZK (P sends nothing and V checks the instance on their own), meaning the claim holds trivially.

Otherwise, suppose $\mathsf{NP} \not\subseteq \mathsf{ioP/poly}$. By Corollary 1, we get that one-way functions exist; we can then use Theorem 11 to amplify the NIZK guaranteed for any $\mathcal{L} \in \mathsf{NP}$ to have negligible errors. □

Corollary 3. *Suppose that* $\mathsf{NP} \not\subseteq \mathsf{ioP/poly}$ *or* $\mathsf{NP} \subseteq \mathsf{BPP}$*. Suppose further that all of* NP *has* $(\epsilon_{\mathsf{c}}, \epsilon_{\mathsf{s}}, \epsilon_{\mathsf{zk}})$*-NIZKs with adaptive soundness where (1)* ϵ_{c} *and* ϵ_{s} *are negligible and (2)* ϵ_{zk} *is a constant smaller than 1. Then all of* NP *has* $(\epsilon'_{\mathsf{c}}, \epsilon'_{\mathsf{s}}, \epsilon'_{\mathsf{zk}})$*-NIZKs with adaptive soundness where* ϵ'_{c}*,* ϵ'_{s}*, and* ϵ'_{zk} *are all negligible.*

Proof. First, suppose $\mathsf{NP} \subseteq \mathsf{BPP}$. In this case, every $\mathcal{L}$ in NP has a trivial NIZK (P sends nothing and V checks the instance on their own), meaning the claim holds trivially.

Otherwise, suppose $\mathsf{NP} \not\subseteq \mathsf{ioP/poly}$. By Theorem 10, we get that one-way functions exist; we can then use Theorem 12 to amplify the NIZK guaranteed for any $\mathcal{L} \in \mathsf{NP}$ to have negligible errors. □

Remark 13. *We note that these amplification results are not entirely unconditional, as they require an assumption that* NP *is either not in* ioP/poly *or is in* BPP*. However, we note that this is a very mild complexity assumption, and so morally these can be viewed as almost unconditional.*

Remark 14. *From Remark 11, we know that if we are willing to use* uniform-secure *one-way functions, we can reduce the requirement that* $\mathsf{NP} \not\subseteq \mathsf{ioP/poly}$ *to* $\mathsf{NP} \not\subseteq \mathsf{ioBPP}$*. Thus, if one could make the theorems from [BG24] work with* uniform-secure *one-way functions, plugging them into our results would only require the assumption that* $\mathsf{NP} \not\subseteq \mathsf{ioBPP}$ *or* $\mathsf{NP} \subseteq \mathsf{BPP}$*. We leave it to future work to determine if this is possible.*

Acknowledgments

J. Hulett and D. Khurana were supported in part by NSF CAREER CNS-2238718 and an award from Visa Research. Also supported in part by DARPA SIEVE award under contract number HR001120C0024. Any opinions, findings and conclusions or recommendations expressed in this material are those of the author(s) and do not necessarily reflect the views of the United States Government or DARPA.

[15] We note that concurrent work [AK25] showed how to improve Theorem 11 to not require that ϵ_{s} and ϵ_{zk} are constants, and in fact work as long as $\epsilon_{\mathsf{s}}(n) + \epsilon_{\mathsf{zk}}(n) <_n 1$. In an earlier version of this paper, we mistakenly claimed that we could use this improved result to get a version of Corollary 2 with similarly relaxed requirements. However, this doesn't quite work, since Corollary 1 (and more directly Lemma 7) requires ϵ_{s} and ϵ_{zk} to be constants.

References

[AK25] Benny Applebaum and Eliran Kachlon. NIZK amplification via leakage-resilient secure computation. Cryptology ePrint Archive, Paper 2025/995, 2025.

[BG24] Nir Bitansky and Nathan Geier. Amplification of non-interactive zero knowledge, revisited. In Leonid Reyzin and Douglas Stebila, editors, *Advances in Cryptology - CRYPTO 2024 - 44th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 18-22, 2024, Proceedings, Part IX*, volume 14928 of *Lecture Notes in Computer Science*, pages 361–390. Springer, 2024.

[BKP+24] Nir Bitansky, Chethan Kamath, Omer Paneth, Ron D. Rothblum, and Prashant Nalini Vasudevan. Batch proofs are statistically hiding. In Bojan Mohar, Igor Shinkar, and Ryan O'Donnell, editors, *Proceedings of the 56th Annual ACM Symposium on Theory of Computing, STOC 2024, Vancouver, BC, Canada, June 24-28, 2024*, pages 435–443. ACM, 2024.

[GJS19] Vipul Goyal, Aayush Jain, and Amit Sahai. Simultaneous amplification: The case of non-interactive zero-knowledge. In Alexandra Boldyreva and Daniele Micciancio, editors, *Advances in Cryptology - CRYPTO 2019 - 39th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 18-22, 2019, Proceedings, Part II*, volume 11693 of *Lecture Notes in Computer Science*, pages 608–637. Springer, 2019.

[HILL99] Johan Håstad, Russell Impagliazzo, Leonid A. Levin, and Michael Luby. A pseudorandom generator from any one-way function. *SIAM Journal on Computing*, 28(4):1364–1396, 1999.

[HN24] Shuichi Hirahara and Mikito Nanashima. One-way functions and zero knowledge. In Bojan Mohar, Igor Shinkar, and Ryan O'Donnell, editors, *Proceedings of the 56th Annual ACM Symposium on Theory of Computing, STOC 2024, Vancouver, BC, Canada, June 24-28, 2024*, pages 1731–1738. ACM, 2024.

[IL89] R. Impagliazzo and M. Luby. One-way functions are essential for complexity based cryptography. In *30th Annual Symposium on Foundations of Computer Science*, pages 230–235, 1989.

[IL90] R. Impagliazzo and L.A Levin. No better ways to generate hard np instances than picking uniformly at random. In *Proceedings [1990] 31st Annual Symposium on Foundations of Computer Science*, pages 812–821 vol.2, 1990.

[Imp92] Russell Impagliazzo. *Pseudo-random generators for cryptography and for randomized algorithms*. PhD thesis, University of California, Berkeley, 1992.

[LMP24] Yanyi Liu, Noam Mazor, and Rafael Pass. A note on zero-knowledge for NP and one-way functions. Cryptology ePrint Archive, Paper 2024/800, 2024.

[Ost91] Rafail Ostrovsky. One-way functions, hard on average problems, and statistical zero-knowledge proofs. In *Proceedings of the Sixth Annual Structure in Complexity Theory Conference, Chicago, Illinois, USA, June 30 - July 3, 1991*, pages 133–138. IEEE Computer Society, 1991.

[OW93] Rafail Ostrovsky and Avi Wigderson. One-way fuctions are essential for non-trivial zero-knowledge. In *Second Israel Symposium on Theory of Computing Systems, ISTCS 1993, Natanya, Israel, June 7-9, 1993, Proceedings*, pages 3–17. IEEE Computer Society, 1993.

[Yao82] Andrew C. Yao. Theory and application of trapdoor functions. In *23rd Annual Symposium on Foundations of Computer Science (sfcs 1982)*, pages 80–91, 1982.

Supplementary material

Disclaimer

Case studies, comparisons, statistics, research and recommendations are provided "AS IS" and intended for informational purposes only and should not be relied upon for operational, marketing, legal, technical, tax, financial or other advice. Visa Inc. neither makes any warranty or representation as to the completeness or accuracy of the information within this document, nor assumes any liability or responsibility that may result from reliance on such information. The Information contained herein is not intended as investment or legal advice, and readers are encouraged to seek the advice of a competent professional where such advice is required.

These materials and best practice recommendations are provided for informational purposes only and should not be relied upon for marketing, legal, regulatory or other advice. Recommended marketing materials should be independently evaluated in light of your specific business needs and any applicable laws and regulations. Visa is not responsible for your use of the marketing materials, best practice recommendations, or other information, including errors of any kind, contained in this document.

A Proof of Theorem 7

For completeness, we here provide a proof of Theorem 7. We note that the proof is essentially the same as that for Theorem 3.9 in [BG24]; our main observation is that because we only care about *non-adaptive* soundness, we the adversary in our reduction can in fact break standard soundness instead of only being able to break soundness*. Before getting into the proof, we first recall a key lemma used in the original; we will use this lemma unchanged.

Lemma 8 ([BG24] Lemma 3.1, adapted from AND Easy-Subset Lemma of [Yao82])**.** *Let $f : \mathcal{X}^k \to \{0,1\}$ be any boolean mapping (possibly randomized), where $\mathcal{X}$ is some finite set and $k \in \mathbb{N}$. Also, let $D : \mathcal{X} \to [0,1]$ be an distribution over $\mathcal{X}$. For any $q \in \mathbb{N}$ and $\epsilon \in (0,1)$*

such that

$$\Pr_{\forall i\in[k]:x_i\leftarrow D}[f(x_1,\ldots,x_k)=1]\geq \epsilon^k+\frac{k}{q},$$

there exists some i *and* $G_i\subseteq\mathcal{X}$ *with* $D(G_i)\geq\epsilon$*, such that for every* $x_i\in G_i$*:*

$$\Pr_{\forall j\in[k]\setminus i:x_j\leftarrow D}[f(x_1,\ldots,x_k)=1]\geq\frac{1}{q}$$

Proof of Theorem 7. First, we note that the proof of zero-knowledge in [BG24] makes no reference to the notion of soundness, and hence goes through exactly as originally written. Thus, it suffices for us to just reprove soundness.

Suppose for the sake of contradiction that there exists an adversary $\mathcal{A}'$ that breaks the soundness of $(\mathrm{Gen}',\mathrm{P}',\mathrm{V}')$. In particular, suppose there exists a polynomial q such that for infinitely many n, there exists $x\in\{0,1\}^n-\mathcal{L}$ with

$$\Pr\left[\mathrm{V}'(\mathsf{crs}',x,\pi')=1\middle|\begin{matrix}\mathsf{crs}'\xleftarrow{\$}\mathrm{Gen}'(1^n)\\ \pi'\leftarrow\mathcal{A}'(\mathsf{crs},x)\end{matrix}\right]>\epsilon_{\mathsf{s}}^k+\frac{2k}{q(n)}\geq\left(\epsilon_{\mathsf{s}}+\frac{1}{q(n)}\right)^k+\frac{k}{q(n)}$$

By applying Lemma 8,[16] we have that there exists an index i^* such that with probability at least $\epsilon_{\mathsf{s}}+\frac{1}{q(n)}$ over sampling crs_{i^*} from $\mathrm{Gen}(1^n)$,

$$\Pr\left[\mathrm{V}'(\mathsf{crs}',x,\pi')=1\middle|\begin{matrix}\mathsf{crs}_j\xleftarrow{\$}\mathrm{Gen}(1^n)\forall j\neq i^*\\ \mathsf{crs}':=(\mathsf{crs}_1,\ldots,\mathsf{crs}_k)\\ \pi'\leftarrow\mathcal{A}'(\mathsf{crs}',x)\end{matrix}\right]\geq\frac{1}{q(n)}\tag{7}$$

This gives rise to a natural adversary $\mathcal{A}$ for $(\mathrm{Gen},\mathrm{P},\mathrm{V})$, given in Algorithm 11.

To analyze $\mathcal{A}$, note first that it always either breaks soundness or outputs $\perp$,[17] so it suffices to upper bound the probability of it never reaching line 7. If the crs input to $\mathcal{A}$ satisfies (7), we know that when $i=i^*$, each run of $\mathcal{A}'$ has probability at least $\frac{1}{q(n)}$ of outputting $\pi'=(\pi_1,\ldots,\pi_k)$ that fools V', which in particular means that π_{i^*} will fool V. Since we run $q(n)\log q(n)$ iterations at this value of i, we have that the probability of never reaching line 7 is at most $\left(1-\frac{1}{q(n)}\right)^{q(n)\log q(n)}<\frac{1}{q(n)}$. Finally, applying that crs satisfies (7) with probability at least $\epsilon_{\mathsf{s}}+\frac{1}{q(n)}$, we have that $\mathcal{A}$ breaks soundness with probability strictly greater than ϵ_{s}, which is a contradiction.

□

[16] In particular, we will apply the lemma with $\mathcal{X}$ as the support of Gen, f as the function that on input $\mathsf{crs}'=(\mathsf{crs}_1,\ldots,\mathsf{crs}_k)$ computes $\pi'=\mathcal{A}'(\mathsf{crs}',x)$ and outputs $\mathrm{V}'(\mathsf{crs}',x,\pi')$, and D as the distribution induced by $\mathrm{Gen}(1^n)$.

[17] Note that this is the critical step where we change the proof from [BG24]. In particular, because the original proof worked with *adaptive* soundness, just seeing an accepting proof isn't enough to break soundness, as the adversary could have selected an $x\in\mathcal{L}$. Because we work with *non-adaptive* soundness, x is fixed and known to not be in $\mathcal{L}$, so any accepting proof we get from $\mathcal{A}'$ is sufficient.

Algorithm 11: Adversary $\mathcal{A}$ for $(\mathrm{Gen}, \mathrm{P}, \mathrm{V})$

Input: $x \in \{0,1\}^n$, $\mathsf{crs} \in \{0,1\}^*$
Output: Proof π for $x \in \mathcal{L}$ under $(\mathrm{Gen}, \mathrm{P}, \mathrm{V})$

for $i \in [k]$ **do**
  repeat $q(n) \log q(n)$ **times**
    For all $j \neq i$, sample $\mathsf{crs}_j \leftarrow \mathrm{Gen}(1^n)$
    Set $\mathsf{crs}_i = \mathsf{crs}$
    Compute $\pi \leftarrow \mathcal{A}'((\mathsf{crs}_1, \ldots, \mathsf{crs}_k), x)$ parsed as $(\pi_1, \ldots \pi_k)$
    if $\mathrm{V}(\mathsf{crs}, x, \pi_i) = 1$ **then**
      Output π_i
    end
  end
end
Output $\perp$

B Proof of Lemma 1

For completeness, we here provide a proof of Lemma 1. We note that this proof is essentially the same as that in [Imp92]; we have mostly rephrased it to fit with our definitions and use case.

Proof of Lemma 1. Let $\{H_n\}_n$ and $\tilde{M}^{-1}$ be as defined in the proof of Theorem 8. We define the N_C as Algorithm 12.

Algorithm 12: Coarse estimator N_C

Input: $x \in \Sigma^n$, $y \in \{0,1\}^{m_o(n)}$
Output: $\tilde{k}_y$, estimate of $|\{r \in \{0,1\}^{m_i(n)} | M(x;r) = y\}|$

Let $i^* = 0$
for $i \leftarrow 1$ **to** $2m_i(n)$ **do**
  for $j \leftarrow 1$ **to** n **do**
    Sample $h \xleftarrow{\$} H_n$ and $z \xleftarrow{\$} \{0,1\}^i$
    Compute $\tilde{r} = \tilde{M}^{-1}(x, h, i, y, z)$
    if $M(x;\tilde{r}) = y$ ***and*** $h(\tilde{r})[:i] = z$ **then**
      Set $i^* = i$
    end
  end
end
Output $\tilde{k}_y = 2^{i^*}$

To prove that this satisfies the conditions of Lemma 1, fix some n such that (5) holds and some $x \in \Sigma^n$. We first note that in (5), we can swap the order of sampling y and r without changing the distribution; that is, we can first get a random y from $M(x)$, then

select r uniformly from the set $R_y := \{r | M(x;r) = y\}$. Applying a Markov bound then tells us that

$$\Pr\left[\Pr\left[\begin{array}{l} M(x;\tilde{r}) = y \\ h(\tilde{r})[:i] = z \end{array}\middle| \begin{array}{c} r \xleftarrow{\$} R_y \\ h \xleftarrow{\$} H_n \\ i \xleftarrow{\$} [2m_i(n)] \\ z \leftarrow h(r)[:i] \\ \tilde{r} \leftarrow \tilde{M}^{-1}(x,h,i,y,z) \end{array}\right] \geq 1 - \frac{\alpha(n)}{12 \cdot m_i(n)} \middle| y \leftarrow M(x)\right] \geq 1 - \frac{1}{12p_1(n)}$$

Let y be "good" if it satisfies the above. Since there are only $2m_i(n)$ possible values that i can take on, as long as y is good we know that for any fixed i,

$$\Pr\left[\begin{array}{l} M(x;\tilde{r}) = y \\ h(\tilde{r})[:i] = z \end{array}\middle| \begin{array}{c} r \xleftarrow{\$} R_y \\ h \xleftarrow{\$} H_n \\ z \leftarrow h(r)[:i] \\ \tilde{r} \leftarrow \tilde{M}^{-1}(x,h,i,y,z) \end{array}\right] \geq 1 - \frac{\alpha(n)}{6} \geq \frac{5}{6} \tag{8}$$

For any distinct $r, r' \in R_y$, pairwise independence of h tells us that $h(r)[:i] = h(r')[:i]$ with probability at most 2^{-i} over $h \xleftarrow{\$} H_n$. Thus, for any $r \in R_y$, the probability over h that there exists such an r' is at most $k_y \cdot 2^{-i}$; if we fix $i = \lceil \log k_y \rceil + 1$, this is at most $\frac{1}{2}$. Thus, for a good y and this choice of i, we have that

$$\Pr\left[\begin{array}{c} M(x;\tilde{r}) = y \\ h(\tilde{r})[:i] = z \\ \nexists r' \neq r \in R_y \text{ s.t. } h(r')[:i] = z \end{array}\middle| \begin{array}{c} r \xleftarrow{\$} R_y \\ h \xleftarrow{\$} H_n \\ z \leftarrow h(r)[:i] \\ \tilde{r} \leftarrow \tilde{M}^{-1}(x,h,i,y,z) \end{array}\right] \geq \frac{1}{3}$$

which with a further Markov bound becomes

$$\Pr\left[\Pr\left[\begin{array}{c} M(x;\tilde{r}) = y \\ h(\tilde{r})[:i] = z \\ \nexists r' \neq r \in R_y \text{ s.t. } h(r')[:i] = z \end{array}\middle| \begin{array}{c} r \xleftarrow{\$} R_y \\ z \leftarrow h(r)[:i] \\ \tilde{r} \leftarrow \tilde{M}^{-1}(x,h,i,y,z) \end{array}\right] \geq \frac{1}{6} \middle| h \xleftarrow{\$} H_n\right] \geq \frac{1}{5} \tag{9}$$

Let h be "good" if it satisfies the above. Define $U_{y,h}$ as the set of all $r \in R_y$ such that there does not exist an $r' \neq r \in R_y$ with $h(r')[:i] = h(r)[:i]$. Noting that the event in (9) cannot happen if $r \notin U_{y,h}$, the probability of its occurring can only increase if we sample $r \xleftarrow{\$} U_{y,h}$. Once we've made this change, we can drop the requirement that there doesn't exist an r' that "collides with r. Thus, for a good h, we have that

$$\Pr\left[\begin{array}{l} M(x;\tilde{r}) = y \\ h(\tilde{r})[:i] = z \end{array}\middle| \begin{array}{c} r \xleftarrow{\$} U_{y,h} \\ z \leftarrow h(r)[:i] \\ \tilde{r} \leftarrow \tilde{M}^{-1}(x,h,i,y,z) \end{array}\right] \geq \frac{1}{6}$$

But note that (9) also tells us that for a good h, $|U_{y,h}| \geq \frac{1}{6}k_y$, since the event therein can only happen if the r we sample from R_y lands in $U_{y,h}$. Since each $r \in U_{y,h}$ has a unique value

of $h(r)[:i]$, we can rewrite this as

$$\Pr\left[\begin{matrix}M(x;\tilde{r}) = y \\ h(\tilde{r})[:i] = z\end{matrix}\middle|\begin{matrix}z \xleftarrow{\$} Z_{y,h} \\ \tilde{r} \leftarrow \tilde{M}^{-1}(x,h,i,y,z)\end{matrix}\right] \geq \frac{1}{6}$$

where $Z_{y,h} := \{h(r)[:i] | r \in U_{y,h}\}$. This in particular means that

$$\begin{aligned}\Pr\left[\begin{matrix}M(x;\tilde{r}) = y \\ h(\tilde{r})[:i] = z\end{matrix}\middle|\begin{matrix}z \xleftarrow{\$} \{0,1\}^i \\ \tilde{r} \leftarrow \tilde{M}^{-1}(x,h,i,y,z)\end{matrix}\right] &\geq \frac{|Z_{y,h}|}{2^i} \cdot \Pr\left[\begin{matrix}M(x;\tilde{r}) = y \\ h(\tilde{r})[:i] = z\end{matrix}\middle|\begin{matrix}z \xleftarrow{\$} Z_{y,h} \\ \tilde{r} \leftarrow \tilde{M}^{-1}(x,h,i,y,z)\end{matrix}\right] \\ &\geq \frac{1}{24} \cdot \frac{1}{6} = \frac{1}{144}\end{aligned}$$

where we used that $|Z_{y,h}| = |U_{y,h}| \geq \frac{1}{6}k_y$ and $2^i \leq 4k_y$.

Putting this all together, we know that as long as y is good, each iteration of the inner loop for $i = \lceil \log k_y \rceil + 1$ chooses a good h with probability least $\frac{1}{5}$ and conditioned on that passes the check on line 6 with probability at least $\frac{1}{144}$. Thus, some iteration passes that check with probability at least $1 - \left(\frac{719}{720}\right)^n$, which for sufficiently large n is at least $1 - \frac{1}{100p_1(n)}$. Noting that our output is at least $2^i > k_y$ as long as at least one of these checks pass, we have that

$$\Pr\left[N_C(x, M(x;r)) < k_{M(x;r)} | r \xleftarrow{\$} \{0,1\}^{m_i(n)}\right] \leq \frac{1}{12p_1(n)} + \frac{1}{100p_1(n)} \leq \frac{1}{9p(n)}$$

for sufficiently large n such that (5) holds and $x \in \Sigma^n$, satisfying the first condition of Lemma 1.

To prove that the second condition holds, fix some $y \in \{0,1\}^{m_o(n)}$ and polynomial q. We will prove that for any $i \geq \log(k_y \cdot q(n))$, the probability of any iteration of the inner loop passing the check on line 6 is sufficiently low. Note that because $|R_y| = k_y$, there are at most k_y values of z such that $h(r)[:i] = z$ for some $r \in R_y$. Thus, the probability that we sample such a z in any given iteration of the inner loop is at most $\frac{k_y}{2^i}$, which is at most $\frac{1}{q(n)}$ since $i \geq \log(k_y \cdot q(n))$. Taking a union bound over at most $2m_i(n)$ valid values of i and n inner loop iterations for each, we have that the probability that $i^* \geq \log(k_y \cdot q(n))$ is at most $\frac{2n \cdot m_i(n)}{q(n)}$. Noting that the output of N_C is 2^{i^*} immediately gives us that we satisfy the second condition of Lemma 1.

□